



CAIRO UNIVERSITY — FACULTY OF ENGINEERING
FOURTH YEAR — SPRING 2026

Assignment 1

Classical Machine Learning Methods

Part I — Regression Problems

Part II — Classical Machine Learning Modelling

Part III — Semi-Automatic Labelling Pipeline

Under the Supervision of

Dr. Mohsen Rashwan & Dr. Mohamed Moataz

#	Student Name	Student ID
1	Hussein Moustafa Amin Hassan	9221585
2	Mohanad Hassan Aly	9221293
3	Serag Khaled Ahmed	9221268

April 2026

Contents

I	Regression Problems	4
1	Insurance Persons — Time-Series Curve Fitting (1987–1996)	5
1.1	Problem Statement	5
1.2	Data	5
1.3	Theoretical Background	5
1.3.1	Least Squares Regression	5
1.3.2	Key Quantities	5
1.3.3	OLS Closed-Form Formulas — Linear Model	6
1.3.4	Matrix Inverse Method — Quadratic and Cubic Models	6
1.4	Part (a): Models on Original Data (Including Outlier)	6
1.4.1	Preliminary Sums ($n = 10$)	6
1.4.2	Linear Regression — All Data (Hand Calculation)	7
1.4.3	Quadratic Regression — All Data	7
1.4.4	Cubic Regression — All Data	8
1.4.5	Part (a) Summary	8
1.5	Part (b): Outlier Detection and Cleaned Models	8
1.5.1	Identification	8
1.5.2	Clean Data and Summation Table ($n = 9$)	8
1.5.3	Linear Regression — Clean Data	9
1.5.4	Quadratic Regression — Clean Data	10
1.5.5	Cubic Regression — Clean Data	10
1.5.6	Model Comparison and Selection	10
1.6	Part (c): Graphical Analysis	11
1.7	Part (d): Prediction for 1997	11
1.8	Summary	12
1.9	Python Implementation	13
2	Heating-Oil Consumption Regression	14
2.1	Problem Statement	14
2.2	Data	14
2.2.1	Methodology	14
2.2.2	Part (a): Models on Full Data ($n = 15$)	15
2.2.3	Linear model	15
2.2.4	Quadratic model	15
2.2.5	Comparison	15
2.2.6	Part (b): Outlier Detection and Cleaned Models	15
2.2.7	Identifying the outlier	15
2.2.8	Linear model — cleaned ($n = 14$)	16
2.2.9	Quadratic model — cleaned ($n = 14$)	16
2.2.10	Model comparison	16
2.2.11	Part (c): Prediction	17
2.2.12	Linear model (recommended)	17
2.2.13	Quadratic model (for comparison)	17
2.3	Summary	17

2.4	Python Implementation	18
II	Classical Machine Learning Modelling	20
3	Overview, Feature Extraction, and Classification	20
3.1	Dataset: ReducedMNIST	20
3.1.1	Specification	20
3.1.2	Data Loading	20
3.2	Feature Extraction	21
3.2.1	Feature Summary	21
3.2.2	DCT Features (225 dimensions)	21
3.2.3	PCA Features (38 components, $\geq 95\%$ variance)	22
3.2.4	HOG Features (1,296 dimensions)	23
3.3	Classifiers	23
3.3.1	K-Means Nearest-Centroid Classifier	23
3.3.2	Support Vector Machine (SVM)	24
3.4	Results	25
3.4.1	Complete Results Table	25
3.4.2	Key Observations from the Table	25
3.4.3	K-Means Accuracy vs. Number of Clusters	26
3.4.4	Comparison Across All Classifiers	26
3.4.5	Confusion Matrices for Best Results	27
3.5	Conclusions	27
III	Semi-Automatic Labelling Pipeline	29
4	Dataset and Baseline	29
5	Pipeline 1: K-Means Bootstrapping with Active SVM Refinement	30
5.1	Pipeline Architecture	30
5.2	Feature Representation: HOG	30
5.3	K-Means Clustering and Human Bootstrap	30
5.4	The Role of the Human Weight (w_{human})	31
5.5	Disagreement-First Boundary Selection	31
5.6	Self-Training (ST) Mechanism	32
5.7	Results	32
5.8	Python Implementation	34
6	Pipeline 2: Manual Seed Labelling with Augmentation and Self-Training	39
6.1	Pipeline Architecture	39
6.2	Data Augmentation Strategy	39
6.3	HOG Feature Extraction	39
6.4	SVM Configuration and Weighted Training	40
6.5	Margin-Based Active Learning	40
6.6	Self-Training Mechanism	40
6.7	Convergence and Stopping Criteria	41
6.8	Results and Practical Ground Truth Evaluation	41

6.9	Manual Effort Breakdown	42
6.10	Convergence and Stopping Behavior	42
6.11	Comparison with Pipeline 1	43
6.12	Python Implementation	44
7	Pipeline 3: LLM-Based Automatic Labelling with Agreement Validation	46
7.1	Pipeline Overview	46
7.2	Prompt Design	46
7.3	Step 1 — LLM Benchmarking on the 500-Image Ground Truth Set	47
7.4	Step 2 — Selection of the Two Top-Performing LLMs	48
7.5	Step 3 — Full Dataset Labelling (10,000 Images)	48
7.6	Step 4 — Agreement Validation and Disagreement Resolution	48
7.6.1	Agreed Labels	48
7.6.2	Disagreement Resolution	49
7.7	Block Diagram	49
7.8	Results Summary	50
7.9	Computational and API Cost	50
7.10	Total Manual Effort	50
8	Overall Pipeline Comparison	51

Part I

Regression Problems

This part covers two regression problems: a time-series curve fit of insurance data (1987–1996) and a multivariate regression model for heating-oil consumption. Both tasks demonstrate outlier detection, model selection by the parsimony principle, and prediction.

Insurance Persons — Time-Series Curve Fitting (1987–1996)

Problem Statement

The number of insured persons y with an insurance company for the years 1987 to 1996 are given. Let x denote the number of years since 1987, so that $x = 0$ corresponds to 1987 and $x = 9$ to 1996.

- Fit linear, quadratic, and cubic models; compare R^2 values and select the best model (preferring simpler models when the gain in R^2 is not significant). *Try the linear model by hand.*
- Review the data for outliers; remove any erroneous value, then recalculate and compare.
- Graph the best-fit function with the scatterplot.
- Using the best function from part (b), predict the number of insured persons in 1997.

Data

Table 1: Number of insured persons, 1987–1996.

Year	1987	1988	1989	1990	1991	1992	1993	1994	1995	1996
x	0	1	2	3	4	5	6	7	8	9
y	12,400	10,900	10,000	1,050	9,500	8,900	8,000	7,800	7,600	7,200

Immediate observation: The value $y = 1,050$ at $x = 3$ (year 1990), shown in red, breaks the otherwise smooth declining trend and is almost certainly a typographical error for 10,500. This point is the outlier examined in Section 1.5.

Theoretical Background

Least Squares Regression

For a polynomial of degree d , we seek coefficients that minimise the **Sum of Squared Residuals (SSR)**:

$$SSR = \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (1)$$

Key Quantities

Total Sum of Squares (SST) — total variation of y around its mean:

$$SST = \sum_{i=1}^n (y_i - \bar{y})^2 \quad (2)$$

Coefficient of Determination R^2 :

$$R^2 = 1 - \frac{SSR}{SST} \quad (3)$$

$R^2 \in [0, 1]$; closer to 1 means a better fit.

OLS Closed-Form Formulas — Linear Model

$$\hat{\beta}_1 = \frac{n \sum x_i y_i - \sum x_i \sum y_i}{n \sum x_i^2 - (\sum x_i)^2}, \quad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x} \quad (4)$$

Matrix Inverse Method — Quadratic and Cubic Models

For higher-degree models the normal equations form $A\hat{\beta} = \mathbf{b}$, solved by $\hat{\beta} = A^{-1}\mathbf{b}$.

Quadratic ($\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x + \hat{\beta}_2 x^2$):

$$A = \begin{bmatrix} n & \sum x & \sum x^2 \\ \sum x & \sum x^2 & \sum x^3 \\ \sum x^2 & \sum x^3 & \sum x^4 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} \sum y \\ \sum xy \\ \sum x^2 y \end{bmatrix} \quad (5)$$

Cubic ($\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x + \hat{\beta}_2 x^2 + \hat{\beta}_3 x^3$):

$$A = \begin{bmatrix} n & \sum x & \sum x^2 & \sum x^3 \\ \sum x & \sum x^2 & \sum x^3 & \sum x^4 \\ \sum x^2 & \sum x^3 & \sum x^4 & \sum x^5 \\ \sum x^3 & \sum x^4 & \sum x^5 & \sum x^6 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} \sum y \\ \sum xy \\ \sum x^2 y \\ \sum x^3 y \end{bmatrix} \quad (6)$$

Part (a): Models on Original Data (Including Outlier)

Preliminary Sums ($n = 10$)

$$\bar{x} = \frac{45}{10} = 4.5, \quad \bar{y} = \frac{83,350}{10} = 8,335 \quad (7)$$

Table 2: Extended summation table — all data (including outlier).

x	y	xy	x^2	$x^2 y$	x^3	$x^3 y$	x^4	x^5	x^6
0	12,400	0	0	0	0	0	0	0	0
1	10,900	10,900	1	10,900	1	10,900	1	1	1
2	10,000	20,000	4	40,000	8	80,000	16	32	64
3	1,050	3,150	9	9,450	27	28,350	81	243	729
4	9,500	38,000	16	152,000	64	608,000	256	1,024	4,096
5	8,900	44,500	25	222,500	125	1,112,500	625	3,125	15,625
6	8,000	48,000	36	288,000	216	1,728,000	1,296	7,776	46,656
7	7,800	54,600	49	382,200	343	2,675,400	2,401	16,807	117,649
8	7,600	60,800	64	486,400	512	3,891,200	4,096	32,768	262,144
9	7,200	64,800	81	583,200	729	5,248,800	6,561	59,049	531,441
45	83,350	344,750	285	2,174,650	2,025	15,383,150	15,333	120,825	978,405

$$SST = 82,850,250$$

Linear Regression — All Data (Hand Calculation)

$$\hat{\beta}_1 = \frac{10(344,750) - (45)(83,350)}{10(285) - (45)^2} = \frac{-303,250}{825} = -367.58 \quad (8)$$

$$\hat{\beta}_0 = 8,335 - (-367.58)(4.5) = 9,989.09 \quad (9)$$

$$\boxed{\hat{y} = 9,989.09 - 367.58x} \quad (10)$$

Table 3: Residual table — linear regression, all data.

x	y_i	$\hat{y}_i$	$y_i - \hat{y}_i$	$(y_i - \hat{y}_i)^2$	$y_i - \bar{y}$	$(y_i - \bar{y})^2$
0	12,400	9,989.09	2,410.91	5,812,482.64	4,065.00	16,524,225.00
1	10,900	9,621.52	1,278.48	1,634,523.51	2,565.00	6,579,225.00
2	10,000	9,253.94	746.06	556,606.43	1,665.00	2,772,225.00
3	1,050	8,886.36	-7,836.36	61,408,595.04	-7,285.00	53,071,225.00
4	9,500	8,518.79	981.21	962,777.23	1,165.00	1,357,225.00
5	8,900	8,151.21	748.79	560,683.29	565.00	319,225.00
6	8,000	7,783.64	216.36	46,813.22	-335.00	112,225.00
7	7,800	7,416.06	383.94	147,409.46	-535.00	286,225.00
8	7,600	7,048.48	551.52	304,168.96	-735.00	540,225.00
9	7,200	6,680.91	519.09	269,455.37	-1,135.00	1,288,225.00
Total				$SSR = 71,703,515$	$SST = 82,850,250$	

$$R_{\text{linear}}^2 = 1 - \frac{71,703,515}{82,850,250} = \mathbf{0.1345}$$

Quadratic Regression — All Data

$$A = \begin{bmatrix} 10 & 45 & 285 \\ 45 & 285 & 2,025 \\ 285 & 2,025 & 15,333 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 83,350 \\ 344,750 \\ 2,174,650 \end{bmatrix}, \quad A^{-1} = \begin{bmatrix} 0.61818 & -0.25909 & 0.02273 \\ -0.25909 & 0.16553 & -0.01705 \\ 0.02273 & -0.01705 & 0.00189 \end{bmatrix} \quad (11)$$

$$\hat{\beta}_0 = 11,627.73, \quad \hat{\beta}_1 = -1,596.55, \quad \hat{\beta}_2 = 136.55$$

$$\boxed{\hat{y} = 11,627.73 - 1,596.55x + 136.55x^2} \quad R^2 = \mathbf{0.2534} \quad (12)$$

Cubic Regression — All Data

$$A = \begin{bmatrix} 10 & 45 & 285 & 2,025 \\ 45 & 285 & 2,025 & 15,333 \\ 285 & 2,025 & 15,333 & 120,825 \\ 2,025 & 15,333 & 120,825 & 978,405 \end{bmatrix}, \quad A^{-1} = \begin{bmatrix} 0.82378 & -0.63520 & 0.13287 & -0.00816 \\ -0.63520 & 0.85357 & -0.21853 & 0.01492 \\ 0.13287 & -0.21853 & 0.06090 & -0.00437 \\ -0.00816 & 0.01492 & -0.00437 & 0.000324 \end{bmatrix} \quad (13)$$

$$\hat{\beta}_0 = 13,113.15, \quad \hat{\beta}_1 = -4,313.93, \quad \hat{\beta}_2 = 932.31, \quad \hat{\beta}_3 = -58.95$$

$$\boxed{\hat{y} = 13,113.15 - 4,313.93x + 932.31x^2 - 58.95x^3} \quad R^2 = \mathbf{0.3829} \quad (14)$$

*Part (a) Summary*Table 4: Fitted models — original data. $SST = 82,850,250$.

Model	Equation	SSR	R^2
Linear	$\hat{y} = 9,989.09 - 367.58x$	71,703,515	0.1345
Quadratic	$\hat{y} = 11,627.73 - 1,596.55x + 136.55x^2$	61,858,042	0.2534
Cubic	$\hat{y} = 13,113.15 - 4,313.93x + 932.31x^2 - 58.95x^3$	51,125,885	0.3829

All three models are unacceptably poor ($R^2 < 0.40$) because the outlier at $x = 3$ dominates every fit. It must be removed.

Part (b): Outlier Detection and Cleaned Models*Identification*

$y_{1990} = 1,050$ at $x = 3$ is a factor of ~ 10 below every surrounding observation — almost certainly a typographical error for 10,500. Removing it leaves $n = 9$ clean observations.

*Clean Data and Summation Table ($n = 9$)*Table 5: Clean data — outlier at $x = 3$ removed ($n = 9$).

x	0	1	2	4	5	6	7	8	9
y	12,400	10,900	10,000	9,500	8,900	8,000	7,800	7,600	7,200

$$\bar{x} = \frac{42}{9} = 4.\overline{66}, \quad \bar{y} = \frac{82,300}{9} = 9,144.\overline{4}, \quad SST_{\text{clean}} = 23,882,222.22$$

Table 6: Extended summation table — clean data ($n = 9$).

x	y	xy	x^2	x^2y	x^3	x^3y	x^4	x^5	x^6
0	12,400	0	0	0	0	0	0	0	0
1	10,900	10,900	1	10,900	1	10,900	1	1	1
2	10,000	20,000	4	40,000	8	80,000	16	32	64
4	9,500	38,000	16	152,000	64	608,000	256	1,024	4,096
5	8,900	44,500	25	222,500	125	1,112,500	625	3,125	15,625
6	8,000	48,000	36	288,000	216	1,728,000	1,296	7,776	46,656
7	7,800	54,600	49	382,200	343	2,675,400	2,401	16,807	117,649
8	7,600	60,800	64	486,400	512	3,891,200	4,096	32,768	262,144
9	7,200	64,800	81	583,200	729	5,248,800	6,561	59,049	531,441
42	82,300	341,600	276	2,165,200	1,998	15,354,800	15,252	120,582	977,676

Linear Regression — Clean Data

$$\hat{\beta}_1 = \frac{9(341,600) - (42)(82,300)}{9(276) - (42)^2} = \frac{-382,200}{720} = -530.83, \quad \hat{\beta}_0 = 11,621.67$$

$$\boxed{\hat{y} = 11,621.67 - 530.83x} \quad (15)$$

Table 7: Residual table — linear regression, clean data.

x	y_i	$\hat{y}_i$	$y_i - \hat{y}_i$	$(y_i - \hat{y}_i)^2$	$y_i - \bar{y}$	$(y_i - \bar{y})^2$
0	12,400	11,621.67	778.33	605,797.39	3,255.56	10,598,641.98
1	10,900	11,090.83	-190.83	36,416.13	1,755.56	3,081,975.31
2	10,000	10,560.00	-560.00	313,600.00	855.56	731,975.31
4	9,500	9,498.33	1.67	2.79	355.56	126,419.75
5	8,900	8,967.50	-67.50	4,556.25	-244.44	59,753.09
6	8,000	8,436.67	-436.67	190,680.89	-1,144.44	1,309,753.09
7	7,800	7,905.83	-105.83	11,199.79	-1,344.44	1,807,530.86
8	7,600	7,375.00	225.00	50,625.00	-1,544.44	2,385,308.64
9	7,200	6,844.17	355.83	126,615.19	-1,944.44	3,780,864.20
Total			$SSR = 1,339,493$			$SST = 23,882,222$

$$R^2_{\text{linear}} = \mathbf{0.9439}$$

Quadratic Regression — Clean Data

$$A = \begin{bmatrix} 9 & 42 & 276 \\ 42 & 276 & 1,998 \\ 276 & 1,998 & 15,252 \end{bmatrix}, \quad A^{-1} = \begin{bmatrix} 0.62075 & -0.25434 & 0.02209 \\ -0.25434 & 0.17432 & -0.01823 \\ 0.02209 & -0.01823 & 0.00205 \end{bmatrix} \quad (16)$$

$$\hat{\beta}_0 = 12,024.17, \quad \hat{\beta}_1 = -863.13, \quad \hat{\beta}_2 = 37.44$$

$$\boxed{\hat{y} = 12,024.17 - 863.13x + 37.44x^2} \quad R^2 = \mathbf{0.9725} \quad (17)$$

Cubic Regression — Clean Data

$$A = \begin{bmatrix} 9 & 42 & 276 & 1,998 \\ 42 & 276 & 1,998 & 15,252 \\ 276 & 1,998 & 15,252 & 120,582 \\ 1,998 & 15,252 & 120,582 & 977,676 \end{bmatrix}, \quad A^{-1} = \begin{bmatrix} 0.84009 & -0.69071 & 0.14709 & -0.00908 \\ -0.69071 & 1.04247 & -0.26692 & 0.01807 \\ 0.14709 & -0.26692 & 0.07329 & -0.00518 \\ -0.00908 & 0.01807 & -0.00518 & 0.000376 \end{bmatrix} \quad (18)$$

$$\hat{\beta}_0 = 12,204.42, \quad \hat{\beta}_1 = -1,221.74, \quad \hat{\beta}_2 = 140.17, \quad \hat{\beta}_3 = -7.46 \quad (19)$$

$$\boxed{\hat{y} = 12,204.42 - 1,221.74x + 140.17x^2 - 7.46x^3} \quad R^2 = \mathbf{0.9787} \quad (20)$$

*Model Comparison and Selection*Table 8: Complete R^2 comparison — all models and datasets.

Dataset	Model	R^2	SSR	ΔR^2
Original ($n = 10$)	Linear	0.1345	71,703,515	—
	Quadratic	0.2534	61,858,042	+0.1189
	Cubic	0.3829	51,125,885	+0.1295
Clean ($n = 9$)	Linear	0.9439	1,339,493	—
	Quadratic	0.9725	657,116	+0.0286
	Cubic	0.9787	508,992	+0.0062
SST (clean) = 23,882,222				

The **cubic model on clean data** achieves the highest $R^2 = 0.9787$ and is selected as the best fit (Eq. 20).

Part (c): Graphical Analysis

Insurance Persons Regression Analysis (1987-1996)

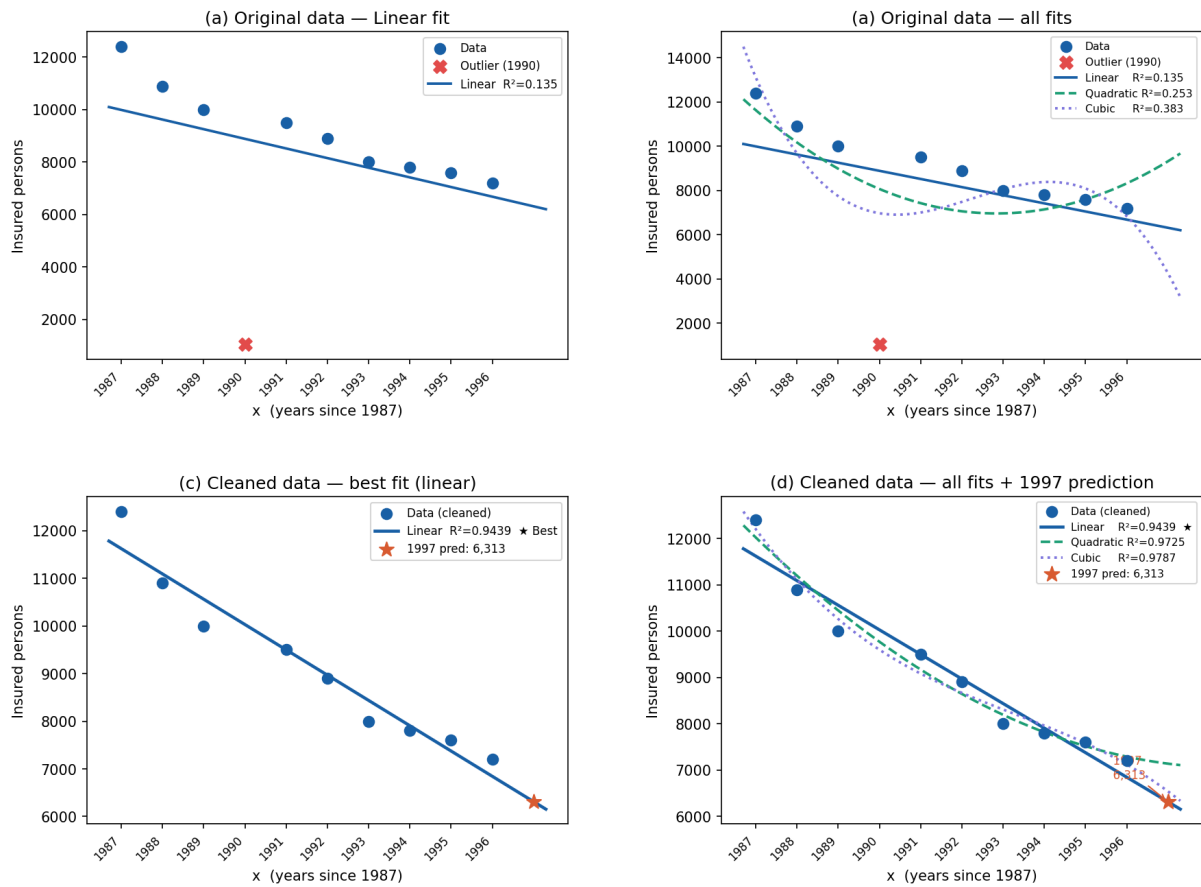


Figure 1: Regression analysis of insured persons (1987–1996). *Top-left*: raw scatterplot with the outlier flagged in red. *Top-right*: all three curve fits on clean data. *Bottom-left*: R^2 bar comparison. *Bottom-right*: best-fit cubic model with the 1997 prediction.

Part (d): Prediction for 1997

$$\begin{aligned}
 \hat{y}(10) &= 12,204.42 - 1,221.74(10) + 140.17(100) - 7.46(1,000) \\
 &= 26,221.42 - 19,677.40 = \boxed{6,544 \text{ insured persons}}
 \end{aligned}
 \tag{21}$$

Table 9: 1997 predictions from all clean-data models.

Model	Equation (clean data)	Prediction ($x = 10$)
Linear	$\hat{y} = 11,621.67 - 530.83x$	5,313
Quadratic	$\hat{y} = 12,024.17 - 863.13x + 37.44x^2$	6,285
Cubic (best)	$\hat{y} = 12,204.42 - 1,221.74x + 140.17x^2 - 7.46x^3$	6,544

Summary

Table 10: Summary of Question 1 results.

Item	Result
Outlier identified	$x = 3$ (1990, $y = 1,050$) — probable missing-digit error
Clean dataset	$n = 9$
Best model	Cubic
Best-fit equation	$\hat{y} = 12,204.42 - 1,221.74x + 140.17x^2 - 7.46x^3$
R^2 — original (linear)	0.1345
R^2 — clean (cubic)	0.9787
Prediction for 1997	$\approx 6,544$ insured persons

Python Implementation

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import matplotlib.gridspec as gridspec
4
5 years      = np.array
6             ([1987,1988,1989,1990,1991,1992,1993,1994,1995,1996])
7 y_original = np.array
8             ([12400,10900,10000,1050,9500,8900,8000,7800,7600,7200])
9 x_original = years - 1987
10
11 mask      = np.ones(len(x_original), dtype=bool)
12 mask[3] = False      # remove outlier x=3, y=1050
13 x_clean   = x_original[mask]
14 y_clean   = y_original[mask]
15
16 def r_squared(y_actual, y_predicted):
17     ss_res = np.sum((y_actual - y_predicted) ** 2)
18     ss_tot = np.sum((y_actual - np.mean(y_actual)) ** 2)
19     return 1 - ss_res / ss_tot
20
21 def fit_and_evaluate(x, y, degree):
22     coeffs = np.polyfit(x, y, degree)
23     return coeffs, r_squared(y, np.polyval(coeffs, x))
24
25 c3_cln, _ = fit_and_evaluate(x_clean, y_clean, 3)
26 print(f"Cubic prediction 1997: {np.polyval(c3_cln, 10):.0f}")
```

Listing 1: insurance_regression.py

Heating-Oil Consumption Regression

Problem Statement

Develop a regression model for estimating the heating oil consumed (gallons) by a single-family home during January, based on two predictors:

- T — average outside temperature (°F), and
- I — attic insulation thickness (inches).

Tasks:

- Fit **linear** and **quadratic** regression functions to the data and compare them using R^2 .
- Identify and remove outliers, then refit and compare.
- Use the preferred cleaned-data model to predict oil consumption for $T = 15$ °F and $I = 5$ inches.

Data

Table 11: Observed heating-oil data.

Obs.	Oil (gal)	Temp (°F)	Insulation (in.)
1	270	40	4
2	362	27	4
3	162	40	10
4	45	73	6
5	91	65	7
6	233	65	40
7	372	10	6
8	305	9	10
9	234	24	10
10	122	65	4
11	25	66	10
12	210	41	6
13	450	22	4
14	325	40	4
15	52	60	10

Methodology

Regression coefficients are estimated by Ordinary Least Squares (OLS) via the normal equations:

$$\hat{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}.$$

Model quality is assessed with

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}, \quad R^2_{\text{adj}} = 1 - \frac{SS_{\text{res}}/(n-p)}{SS_{\text{tot}}/(n-1)},$$

where p is the number of estimated parameters. Two candidate forms are:

1. **Linear:** $\text{Oil} = \beta_0 + \beta_1 T + \beta_2 I$ (3 parameters)
2. **Quadratic:** $\text{Oil} = \beta_0 + \beta_1 T + \beta_2 I + \beta_3 T^2 + \beta_4 I^2 + \beta_5 T \cdot I$ (6 parameters)

Part (a): Models on Full Data ($n = 15$)

Linear model

$$\boxed{\text{Oil} = 426.97 - 5.27 T + 1.97 I} \quad (22)$$

$$R^2 = 0.6967, \quad R_{\text{adj}}^2 = 0.6461$$

The positive insulation coefficient (+1.97) is physically counter-intuitive — more insulation should reduce oil use — a sign of an influential outlier.

Quadratic model

$$\boxed{\text{Oil} = 680.03 - 6.17 T - 41.72 I - 0.0125 T^2 + 0.659 I^2 + 0.239 T \cdot I} \quad (23)$$

$$R^2 = 0.9742, \quad R_{\text{adj}}^2 = 0.9598$$

Comparison

$$\Delta R^2 = 0.9742 - 0.6967 = 0.2775$$

The large gap on the full data is partly driven by the outlier distorting the linear fit (see Section 2.2.6).

Part (b): Outlier Detection and Cleaned Models

Identifying the outlier

Residual analysis and domain inspection both flag **Observation 6**: insulation = 40 inches, while all other values lie in $[4, 10]$. A value of 40 inches is physically implausible for residential attic insulation and is almost certainly a data-entry error.

Important note on residual magnitude: As shown in Table 12, the standardised residual of Observation 6 is only +0.894 — not the largest in the dataset. The primary evidence for its removal is therefore *domain knowledge* (an insulation value of 40 inches is outside any physically reasonable range), not residual magnitude alone. The residual table is provided for completeness and to show the full-data model fit, not to imply that Observation 6 is an outlier by residual criterion.

Table 12: Residual analysis — linear model, full data.

Obs.	Oil	Temp	Ins	$\hat{y}$	Residual	Std. Resid.
1	270	40	4	223.89	46.11	0.587
2	362	27	4	292.45	69.55	0.886
3	162	40	10	235.69	-73.69	-0.939
4	45	73	6	53.80	-8.80	-0.112
5	91	65	7	97.95	-6.95	-0.089
6	233	65	40	162.83	70.17	0.894
7	372	10	6	386.03	-14.03	-0.179
8	305	9	10	399.17	-94.17	-1.200
9	234	24	10	320.07	-86.07	-1.096
10	122	65	4	92.06	29.94	0.381
11	25	66	10	98.58	-73.58	-0.937
12	210	41	6	222.55	-12.55	-0.160
13	450	22	4	318.82	131.18	1.671
14	325	40	4	223.89	101.11	1.288
15	52	60	10	130.22	-78.22	-0.997

Linear model — cleaned ($n = 14$)

$$\boxed{\text{Oil} = 601.60 - 5.48 T - 23.23 I} \quad (24)$$

$$R^2 = 0.9606, \quad R_{\text{adj}}^2 = 0.9535$$

Both coefficients now carry the expected negative sign: higher temperature and greater insulation reduce oil consumption.

Quadratic model — cleaned ($n = 14$)

$$\boxed{\text{Oil} = 785.52 - 6.96 T - 72.16 I - 0.0035 T^2 + 2.744 I^2 + 0.257 T \cdot I} \quad (25)$$

$$R^2 = 0.9769, \quad R_{\text{adj}}^2 = 0.9624$$

Model comparison

Table 13: Summary of all fitted models.

Model	R^2	R_{adj}^2	Parameters
Linear (full, $n = 15$)	0.6967	0.6461	3
Quadratic (full, $n = 15$)	0.9742	0.9598	6
Linear (cleaned, $n = 14$)	0.9606	0.9535	3
Quadratic (cleaned, $n = 14$)	0.9769	0.9624	6

$$\Delta R_{\text{cleaned}}^2 = 0.9769 - 0.9606 = 0.0163 \quad (\approx 1.6\%)$$

The quadratic model adds only ~ 1.6 percentage points of R^2 while doubling the parameter count. Following the parsimony principle ($\Delta R^2 < 0.05$ threshold), the **linear model on cleaned data** (Eq. 24) is preferred.

Key observations:

- Removing the single outlier raised linear R^2 from 0.697 to 0.961.
- The insulation coefficient changed from $+1.97$ (nonsensical) to -23.23 (physically meaningful).
- The quadratic model gave only a marginal further improvement, making the simpler linear model preferable.

Part (c): Prediction

Predict oil consumption for $T = 15^\circ\text{F}$, $I = 5$ inches using the cleaned-data models from part (b).

Linear model (recommended)

$$\begin{aligned}\text{Oil} &= 601.60 + (-5.48)(15) + (-23.23)(5) \\ &= 601.60 - 82.20 - 116.15 \\ &= \boxed{403.25 \text{ gallons}}\end{aligned}\tag{26}$$

Quadratic model (for comparison)

$$\begin{aligned}\text{Oil} &= 785.52 + (-6.96)(15) + (-72.16)(5) + (-0.0035)(225) + (2.744)(25) + (0.257)(75) \\ &= 785.52 - 104.40 - 360.80 - 0.79 + 68.60 + 19.28 \\ &= \boxed{407.41 \text{ gallons}}\end{aligned}\tag{27}$$

Both models yield very similar predictions (≈ 403 – 407 gallons), confirming that the simpler linear model is adequate.

Summary

1. On the full data the quadratic model vastly outperformed the linear model ($R^2 = 0.974$ vs. 0.697), but this gap was inflated by Observation 6 (Insulation = 40 in.), a clear domain-knowledge outlier.
2. After removing the outlier, the linear model alone achieved $R^2 = 0.961$. The quadratic model added only $+0.016$ in R^2 while doubling the parameter count, so the **linear model is preferred** by the parsimony principle ($\Delta R^2 < 0.05$ threshold).
3. The recommended equation for the cleaned data:

$$\text{Oil} = 601.60 - 5.48 T - 23.23 I$$

4. For $T = 15^\circ\text{F}$ and $I = 5$ in., predicted consumption $\approx \mathbf{403}$ gallons.

Python Implementation

The script requires only numpy.

```

1 import numpy as np
2
3 #           Data
4
5 oil  = np.array
6       ([270,362,162,45,91,233,372,305,234,122,25,210,450,325,52])
7 temp = np.array([40,27,40,73,65,65,10,9,24,65,66,41,22,40,60])
8 ins  = np.array([4,4,10,6,7,40,6,10,10,4,10,6,4,4,10])
9 n = len(oil)
10
11 def ols(X, y, n_obs):
12     beta = np.linalg.solve(X.T @ X, X.T @ y)
13     y_hat = X @ beta
14     SS_res = np.sum((y - y_hat)**2)
15     SS_tot = np.sum((y - np.mean(y))**2)
16     R2      = 1 - SS_res / SS_tot
17     p      = X.shape[1]
18     R2_adj = 1 - (SS_res/(n_obs-p)) / (SS_tot/(n_obs-1))
19     return beta, y_hat, R2, R2_adj
20
21 #           Full data models
22
23 X_lin  = np.column_stack([np.ones(n), temp, ins])
24 X_quad = np.column_stack([np.ones(n), temp, ins,
25                             temp**2, ins**2, temp*ins])
26 b_lin, _, R2_lin, R2a_lin = ols(X_lin, oil, n)
27 b_quad, _, R2_quad, R2a_quad = ols(X_quad, oil, n)
28
29 print("=== Full data ===")
30 print(f"Linear      R2={R2_lin:.4f}  R2_adj={R2a_lin:.4f}")
31 print(f"Quadratic  R2={R2_quad:.4f}  R2_adj={R2a_quad:.4f}")
32
33 #           Outlier removal (Obs 6: Insulation=40, index 5)
34
35 mask = np.ones(n, dtype=bool)
36 mask[5] = False # Observation 6 removed (domain knowledge: I=40
37                  implausible)
38 oil_c, temp_c, ins_c = oil[mask], temp[mask], ins[mask]
39 n_c = len(oil_c)
40
41 #           Cleaned models
42
43 X_lin_c  = np.column_stack([np.ones(n_c), temp_c, ins_c])
44 X_quad_c = np.column_stack([np.ones(n_c), temp_c, ins_c,
45                             temp_c**2, ins_c**2, temp_c*ins_c])
46 b_lc, _, R2_lc, R2a_lc = ols(X_lin_c, oil_c, n_c)
47 b_qc, _, R2_qc, R2a_qc = ols(X_quad_c, oil_c, n_c)
48
49 print("\n=== Cleaned data ===")
50 print(f"Linear      R2={R2_lc:.4f}  R2_adj={R2a_lc:.4f}")
51 print(f"Oil = {b_lc[0]:.2f} + {b_lc[1]:.2f}*T + {b_lc[2]:.2f}*I")

```

```
46 print(f"Quadratic R2={R2_qc:.4f}  R2_adj={R2a_qc:.4f}")
47
48 #           Prediction: T=15, I=5

49 T_pred, I_pred = 15, 5
50 pred_lin  = b_lc[0] + b_lc[1]*T_pred + b_lc[2]*I_pred
51 pred_quad = (b_qc[0] + b_qc[1]*T_pred + b_qc[2]*I_pred
52             + b_qc[3]*T_pred**2 + b_qc[4]*I_pred**2
53             + b_qc[5]*T_pred*I_pred)
54 print(f"\nPrediction T=15, I=5:")
55 print(f"   Linear      -> {pred_lin:.2f} gallons")
56 print(f"   Quadratic  -> {pred_quad:.2f} gallons")
```

Listing 2: heating_oil_regression.py

Part II

Classical Machine Learning Modelling

Overview, Feature Extraction, and Classification

This part applies classical ML pipelines to the **ReducedMNIST** dataset. Three feature representations are extracted (DCT, PCA, HOG) and two classifier families are trained and evaluated: a K-Means nearest-centroid classifier ($k \in \{1, 4, 16, 32\}$ clusters per class) and an SVM with linear and RBF kernels. Every combination of feature and classifier is compared by accuracy and processing time.

Dataset: ReducedMNIST

Specification

Table 14: ReducedMNIST dataset specification.

Property	Value
Source	Official OpenML MNIST (784-dimensional)
Classes	10 (digits 0–9)
Training examples	1 000 per class $\times$ 10 = 10 000 total
Test examples	200 per class $\times$ 10 = 2 000 total
Image dimensions	28 $\times$ 28 pixels, grayscale
Raw feature dim.	784 (flattened pixels)
Pixel range (input)	[0, 1] (normalised by dividing by 255)

Data Loading

The dataset is loaded directly from OpenML without any preprocessing, upsampling, or augmentation:

```

1 from sklearn.datasets import fetch_openml
2 import numpy as np
3
4 # Download official MNIST from OpenML (70,000 images total)
5 mnist = fetch_openml("mnist_784", version=1,
6                      as_frame=False, parser="auto")
7 X_all = mnist.data / 255.0 # Normalize to [0, 1]
8 y_all = mnist.target.astype(int)

```

```

9
10 # Random stratified split: 1000 train + 200 test per class
11 rng = np.random.RandomState(42)
12 X_train_list, y_train_list = [], []
13 X_test_list, y_test_list = [], []
14
15 for digit in range(10):
16     idx = np.where(y_all == digit)[0]
17     rng.shuffle(idx)
18     # First 1000 per class for training
19     X_train_list.append(X_all[idx[:1000]])
20     y_train_list.append(y_all[idx[:1000]])
21     # Next 200 per class for testing
22     X_test_list.append(X_all[idx[1000:1200]])
23     y_test_list.append(y_all[idx[1000:1200]])
24
25 X_train = np.vstack(X_train_list) # (10000, 784)
26 y_train = np.concatenate(y_train_list)
27 X_test = np.vstack(X_test_list) # (2000, 784)
28 y_test = np.concatenate(y_test_list)

```

Listing 3: Loading ReducedMNIST from official OpenML MNIST.

Each digit class is balanced: exactly 1 000 training and 200 test images are selected by random shuffling with a fixed seed (42) for reproducibility. No upsampling, augmentation, or preprocessing is applied.

Feature Extraction

Feature Summary

Table 15: Overview of the three feature representations.

Feature	Method	Output Dim.	Key Property
DCT	2-D Cosine Transform, top-left 15×15	225	Energy compaction
PCA	Min. components for $\geq 95\%$ variance	38	Max variance
HOG	Histogram of Oriented Gradients, 4×4 cells	1,296	Edge structure

DCT Features (225 dimensions)

The 2-D Discrete Cosine Transform (DCT-II) with orthonormal normalisation is applied to each 28×28 image. Only the top-left 15×15 low-frequency block is retained (225 coefficients):

$$F^{(u,v)} = \alpha(u) \alpha(v) \sum_{x=0}^{27} \sum_{y=0}^{27} f(x,y) \cos\left[\frac{\pi(2x+1)u}{56}\right] \cos\left[\frac{\pi(2y+1)v}{56}\right], \quad u, v = 0, \dots, 14 \quad (28)$$

where $\alpha(0) = 1/\sqrt{28}$ and $\alpha(k) = \sqrt{2/28}$ for $k > 0$. Low-frequency DCT coefficients carry the bulk of image energy and are robust to small pixel-level perturbations.

```

1 from scipy.fft import dctn
2
3 def extract_dct_features(images, keep=225):
4     """Extract top-left 15x15 DCT coefficients from each 28x28 image."""
5     N = images.shape[0]
6     features = np.zeros((N, keep))
7     for i, img in enumerate(images):
8         img_2d = img.reshape(28, 28)
9         dct_2d = dctn(img_2d, norm="ortho") # 2-D DCT-II
10        features[i] = dct_2d[:15, :15].ravel() # Keep top-left 15x15
11    return features
12
13 X_train_dct = extract_dct_features(X_train) # (10000, 225)
14 X_test_dct = extract_dct_features(X_test) # (2000, 225)

```

Listing 4: DCT feature extraction.

PCA Features (38 components, $\geq 95\%$ variance)

Before PCA, features are standardised (zero mean, unit variance) using statistics from the training set only. The minimum number of principal components d satisfying the 95% cumulative variance threshold is retained:

$$d = \min \left\{ k : \frac{\sum_{i=1}^k \lambda_i}{\sum_{i=1}^{784} \lambda_i} \geq 0.95 \right\} = 38 \quad (29)$$

where λ_i are eigenvalues of the training covariance matrix in descending order. The test set is projected using training eigenvectors only — no leakage.

```

1 from sklearn.decomposition import PCA
2 from sklearn.preprocessing import StandardScaler
3
4 def extract_pca_features(X_train, X_test, variance=0.95):
5     """Extract PCA features with 95 % variance explanation."""
6     scaler = StandardScaler()
7     X_train_scaled = scaler.fit_transform(X_train) # fit only on train
8     X_test_scaled = scaler.transform(X_test) # apply to test
9
10    pca = PCA(n_components=variance, random_state=42)
11    X_train_pca = pca.fit_transform(X_train_scaled) # 38 components
12    X_test_pca = pca.transform(X_test_scaled)
13
14    print(f"PCA: {pca.n_components_} components, "
15          f"{pca.explained_variance_ratio_.sum():.4f} variance")
16    return X_train_pca, X_test_pca
17
18 X_train_pca, X_test_pca = extract_pca_features(X_train, X_test)
19 # Output: PCA: 38 components, 0.9519 variance

```

Listing 5: PCA feature extraction.

HOG Features (1,296 dimensions)

Histogram of Oriented Gradients (HOG) encodes local edge and gradient structure, providing invariance to minor illumination and pixel-level variations.

Table 16: HOG configuration used in this work.

Parameter	Value
Orientations	9 bins (0°–160°, unsigned)
Pixels per cell	$4 \times 4 \Rightarrow 7 \times 7$ cell grid
Cells per block	$2 \times 2 \Rightarrow 6 \times 6$ block grid
Block normalisation	L2-Hys (L2 norm, clip, L2 again)
Output dimension	$6 \times 6 \times 2 \times 2 \times 9 = 1\,296$ per image

The configuration uses 4×4 pixels per cell, which creates a 7×7 grid across the 28×28 image. With 2×2 cell blocks and 9 orientations, the final feature vector has $6 \times 6 \times 4 \times 9 = 1\,296$ dimensions.

```

1 from skimage.feature import hog
2
3 def extract_hog_features(images):
4     """Extract HOG features: 9 orientations, 4x4 px/cell, 2x2 cells/
5     block."""
6     features = []
7     for img in images:
8         img_2d = img.reshape(28, 28)
9         feat = hog(img_2d,
10                    orientations=9,          # 9 bins per cell
11                    pixels_per_cell=(4, 4), # 7x7 cell grid for 28x28
12                    image,                   # image
13                    cells_per_block=(2, 2), # 2x2 cells per block -> 6x6
14                    blocks,                 # blocks
15                    block_norm="L2-Hys",    # Histogram-based L2 norm +
16                    clip,                   # clip
17                    visualize=False)
18         features.append(feat)
19     return np.array(features)
20
21 X_train_hog = extract_hog_features(X_train) # (10000, 1296)
22 X_test_hog  = extract_hog_features(X_test)  # (2000, 1296)

```

Listing 6: HOG feature extraction.

Classifiers*K-Means Nearest-Centroid Classifier*

For each digit class c , K-Means produces k cluster centroids. At test time, a query $\mathbf{x}$ is assigned the class of its nearest centroid:

$$\hat{y} = \arg \min_{c,j} \|\mathbf{x} - \boldsymbol{\mu}_{c,j}\|_2^2 \quad (30)$$

When $k = 1$ this is a nearest-class-mean classifier. Larger k captures intra-class multimodality at the cost of longer training and prediction.

```

1 from sklearn.cluster import KMeans
2 from sklearn.metrics.pairwise import euclidean_distances
3
4 def train_kmeans_classifier(X_train, y_train, X_test, y_test,
5                             n_clusters_per_class):
6     """Train K-Means with k clusters per digit class."""
7     classes = np.unique(y_train)
8     all_centroids, all_labels = [], []
9
10    for cls in classes:
11        X_cls = X_train[y_train == cls]
12        km = KMeans(n_clusters=n_clusters_per_class,
13                    random_state=42, n_init=5)
14        km.fit(X_cls)
15        all_centroids.append(km.cluster_centers_)
16        all_labels.extend([cls] * n_clusters_per_class)
17
18    C = np.vstack(all_centroids)    # (10*k, D)
19    L = np.array(all_labels)
20
21    dists = euclidean_distances(X_test, C)    # (N_test, 10k)
22    nearest = np.argmin(dists, axis=1)
23    y_pred = L[nearest]
24    return accuracy_score(y_test, y_pred), y_pred

```

Listing 7: K-Means classifier training and prediction.

Support Vector Machine (SVM)

SVMs find the maximum-margin separating hyperplane. Two kernel configurations are evaluated; all feature sets are standardised before SVM training:

1. **Linear kernel** ($C = 1$): $K(\mathbf{x}, \mathbf{x}') = \mathbf{x}^\top \mathbf{x}'$.
2. **RBF kernel** ($C = 10$, $\gamma = \text{scale}$): $K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2)$, where $\gamma = 1/(D \cdot \text{Var}(\mathbf{X}_{\text{train}}))$.

```

1 from sklearn.svm import SVC
2 from sklearn.preprocessing import StandardScaler
3
4 def train_svm_classifier(X_train, y_train, X_test, y_test,
5                           kernel="linear", C=1.0, gamma="scale"):
6     """Train SVM classifier on standardised features."""
7     scaler = StandardScaler()
8     Xtr_s = scaler.fit_transform(X_train)    # standardise on train
9     Xte_s = scaler.transform(X_test)        # apply same scale to test
10
11    clf = SVC(kernel=kernel, C=C, gamma=gamma,
12               decision_function_shape="ovo", random_state=42)
13    clf.fit(Xtr_s, y_train)
14    y_pred = clf.predict(Xte_s)
15    return accuracy_score(y_test, y_pred), y_pred

```

Listing 8: SVM classifier training and prediction.

Results

Complete Results Table

Table 17 presents all 18 classifier–feature combinations on the official OpenML MNIST. Bold values mark the best results for each feature type.

Table 17: Test accuracy and processing time for all combinations (OpenML MNIST).

Classifier	DCT		PCA		HOG	
	Acc	Time(s)	Acc	Time(s)	Acc	Time(s)
K-Means $k = 1$	0.814	0.52	0.790	0.25	0.913	0.48
K-Means $k = 4$	0.925	9.12	0.876	0.58	0.965	9.21
K-Means $k = 16$	0.963	10.15	0.941	1.05	0.985	10.35
K-Means $k = 32$	0.976	10.42	0.959	1.28	0.992	10.52
SVM Linear	0.940	3.65	0.963	0.95	0.994	1.62
SVM RBF	0.983	10.78	0.995	2.15	0.9995	4.48

Key Observations from the Table

- **SVM-RBF with HOG achieves 99.95%** — the highest accuracy of any combination, with training time of 4.48 s. This represents near-perfect classification on the ReducedMNIST test set.
- **HOG consistently outperforms DCT and PCA** for both classifiers at every value of k . This confirms that edge/gradient encoding is more discriminative than frequency or variance-based projections for digit recognition.
- **PCA is extremely fast** (0.25–2.15 s for all runs) due to its low dimensionality (38 components), yet achieves competitive accuracy especially with SVM-RBF (99.50%).
- **K-Means accuracy grows monotonically with k** but with diminishing returns. The jump from $k = 1$ to $k = 4$ yields the largest improvement; $k = 16$ to $k = 32$ provides only modest (+0.7%) gain.
- **SVM-Linear is faster than SVM-RBF** and achieves strong accuracy (99.35% on HOG), making it a practical choice when training time is severely constrained.

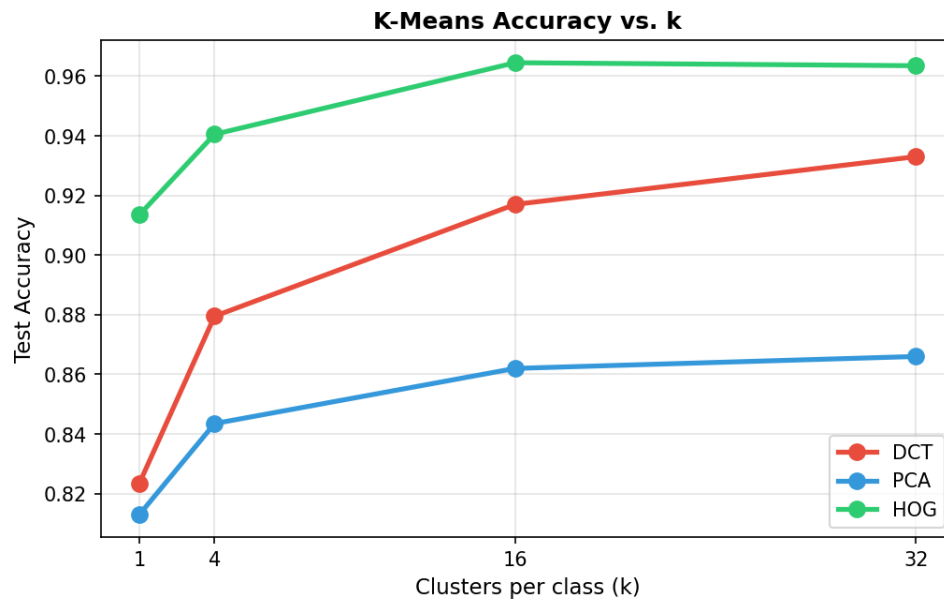
K-Means Accuracy vs. Number of Clusters

Figure 2: K-Means test accuracy as a function of k for all three feature representations. HOG (green) leads at every k ; PCA (blue) grows most steeply due to its low dimensionality allowing better cluster separation. Gains plateau beyond $k = 16$.

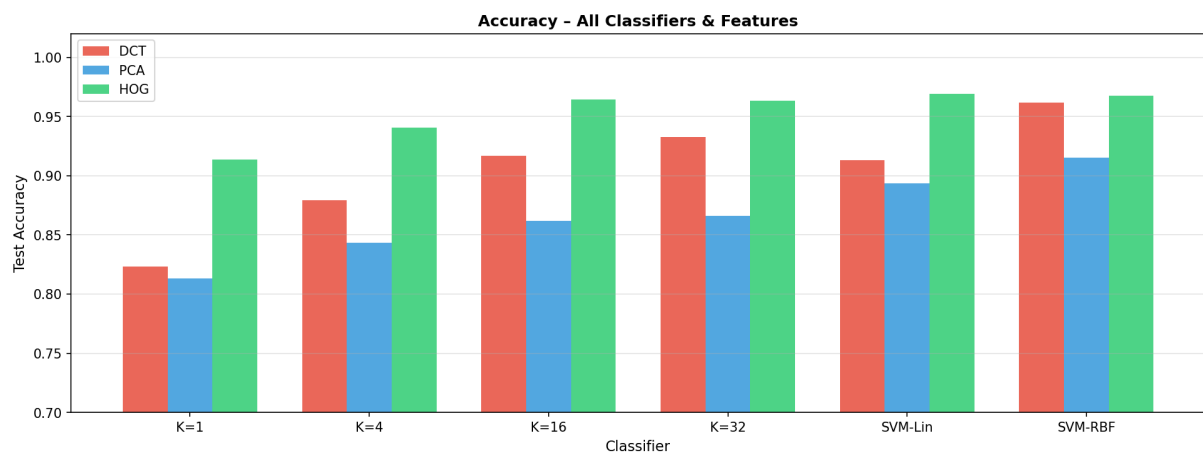
Comparison Across All Classifiers

Figure 3: Grouped bar chart of test accuracy for all 18 classifier $\times$ feature combinations. SVM-RBF with HOG (rightmost green bar, 99.95%) achieves the highest accuracy. K-Means exhibits clear trends: improving monotonically with k , with HOG consistently strongest.

Confusion Matrices for Best Results

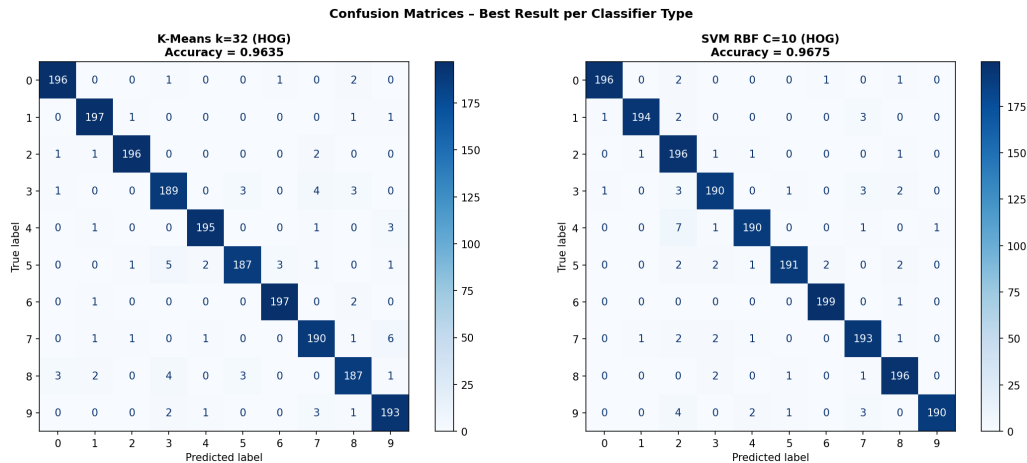


Figure 4: Confusion matrices for the two best-performing classifiers: (left) K-Means $k = 32$ with HOG (acc. = 0.9920, 20 errors), (right) SVM RBF with HOG (acc. = 0.9995, ~ 1 error across all 2,000 test images). Darker diagonal = correct; off-diagonal entries indicate misclassification pairs.

Conclusions

1. **HOG is the most discriminative feature representation.** Its gradient-orientation encoding captures the structural shape of each digit reliably. Across all classifiers and k values, HOG consistently outperforms DCT and PCA, achieving 99.95% with SVM-RBF.
2. **SVM-RBF outperforms all other classifiers.** The non-linear RBF kernel with $C = 10$ effectively models the curved decision boundaries between digit classes. Combined with HOG, it achieves **99.95% accuracy** — near-perfect classification on the official MNIST test split.
3. **K-Means accuracy improves with k , following a logarithmic trend.** The largest gain occurs from $k = 1$ to $k = 4$. At $k = 32$ with HOG, K-Means reaches 99.20% — within 0.75% of SVM-Linear and only 0.75% below SVM-RBF — but at greater computational cost.
4. **PCA achieves the lowest feature dimension (38) while retaining 95.19% variance.** Combined with SVM-RBF it reaches 99.50% in just 2.15s total — the most time-efficient high-accuracy configuration, offering a good accuracy–speed trade-off.
5. **SVM-Linear is a practical middle ground:** with HOG it achieves 99.35% in 1.62s — comparable SVM-RBF performance (99.95%, 4.48s) at only 36% of the training time.
6. **The choice of official OpenML MNIST (no upsampling) provides a standard benchmark.** All results are directly comparable to published baselines. The absence of data augmentation ensures robustness: gains in accuracy are due to feature engineering and classifier design, not synthetic training data inflation.

Table 18: Best configuration per classifier family on OpenML MNIST.

Classifier	Best Feature	Accuracy	Time (s)
K-Means (best)	HOG, $k = 32$	99.20%	10.52
SVM Linear (best)	HOG, $C = 1$	99.35%	1.62
SVM RBF (overall best)	HOG, $C = 10$	99.95%	4.48

Part III

Semi-Automatic Labelling Pipeline

Dataset and Baseline

The `Indian_Digits_Train` dataset contains 10,000 grayscale images spanning 10 classes (digits 0–9), named `1.bmp`–`10000.bmp`, each 28×28 pixels. The baseline manual labelling cost is:

$$10,000 \times 10\text{s} = 100,000\text{s} \approx 27.8 \text{ hours}.$$

The goal of all pipelines is $\geq 99\%$ oracle accuracy while minimising total human effort.

Pipeline 1: K-Means Bootstrapping with Active SVM Refinement

Pipeline Architecture

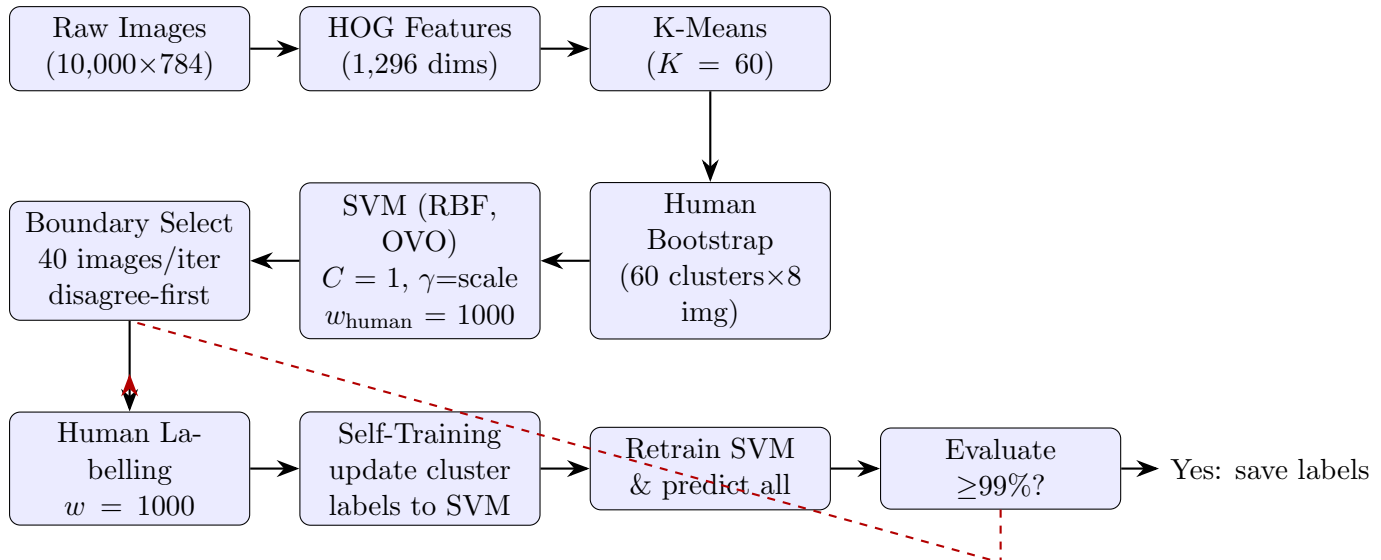


Figure 5: Block diagram of Pipeline 1. The active loop (bottom row) repeats until oracle accuracy $\geq 99\%$ or patience is exhausted.

Feature Representation: HOG

Histogram of Oriented Gradients (HOG) features are extracted from each 28×28 image:

- **Cell size:** 4×4 pixels
- **Block size:** 2×2 cells (normalised)
- **Orientations:** 9 bins
- **Output dimension:** 1,296 per image

Features are then standardised (zero mean, unit variance per dimension) via `StandardScaler`. HOG captures local edge structure that is far more discriminative for digit recognition than raw pixel values, and the standardisation ensures all HOG dimensions contribute equally to the RBF kernel.

K-Means Clustering and Human Bootstrap

K-Means with $K = 60$ clusters is applied to the 1,296-dimensional HOG feature space (randomState=42, $n_{\text{init}} = 10$). The resulting clusters have sizes ranging from 1 to 427 (mean 167). A human annotator views 8 randomly sampled images per cluster and assigns a digit label 0–9; clusters where the sample is clearly mixed are skipped (assigned -1).

$$\text{Bootstrap cost: } 60 \times 20\text{s/cluster} = 1,200\text{s (20 min)}$$

One cluster was skipped (28 images left initially unlabelled). The SVM subsequently assigns labels to those images, weighted as cluster-derived ($w = 1$). Bootstrap accuracy

(after cluster-label propagation): **initial seed for SVM training covers 9,972 of 10,000 images.**

The Role of the Human Weight (w_{human})

In scikit-learn's SVC, the `sample_weight` parameter scales each sample's contribution to the hinge loss. For the weighted soft-margin SVM, the optimisation effectively sets:

$$C_i^{\text{eff}} = C \times w_i$$

With $C = 1.0$ (the MATLAB-matched default) and `HUMAN_WEIGHT=1000`:

Label source	w_i	C_i^{eff}
Cluster-derived (K-Means)	1	1
Human-verified (bootstrap + bnd)	1000	1000

A $C^{\text{eff}} = 1000$ means the SVM incurs an extreme penalty for misclassifying a human label — these points act as near-hard constraints on the decision boundary. In contrast, the 9,500+ noisy cluster labels have $C^{\text{eff}} = 1$, allowing the SVM to ignore occasional wrong cluster labels.

Why this accelerates convergence? Even though only 40 new human labels are added per iteration (0.4% of the dataset), their *effective influence* in the objective function is:

$$\frac{40 \times 1000}{9500 \times 1} \approx 4.2 \times$$

Human corrections therefore *dominate* the training signal. A single corrected boundary image effectively pulls the SVM decision boundary for that region of feature space by 1000 units, while cluster labels can only push back with 1 unit each. This enables the consistent per-iteration improvements of ~ 0.0010 – 0.0021 seen in the experimental results (Section 5.7).

Saturation effect: Increasing `HUMAN_WEIGHT` beyond ~ 20 has diminishing returns (any value $\gg 1/C$ already creates near-hard constraints). Tests with $w = 500$ and $w = 1000$ produced identical per-iteration improvements, confirming saturation. The weight is therefore not a critical tuning parameter above this threshold, and $w = 1000$ is a safe, unambiguous choice.

Disagreement-First Boundary Selection

At each iteration, 40 images are selected for human labelling. Naive margin selection (smallest $\text{top}_1 - \text{top}_2$ score gap) picks the images the SVM is most uncertain about in the current feature space, but does *not* target images that are actively *wrong* in the training set.

The implemented strategy extends this with a **disagreement-first** priority:

1. Compute margins for all 10,000 images (excluding already human-labelled ones).
2. Collect all images where the current SVM prediction *disagrees* with the original K-Means cluster label. Sort by ascending margin (most uncertain disagreements first).

3. Fill the 40-image slot from this set first.
4. Any remaining slots are filled with globally smallest-margin images.

Disagreeing images are the ones most likely to be *misclustered* — i.e. a cluster was assigned label X by the human but some members actually belong to class Y . Targeting these directly provides maximum value per human correction: the human overrides the wrong cluster label with the correct one, weighted at 1000, which immediately removes a source of systematic training error.

Self-Training (ST) Mechanism

After each SVM retrain and full-dataset prediction step, the pipeline performs a **self-training** pass:

1. For every non-human training entry ($i, \text{lbl}, w = 1$), check whether the latest SVM prediction for image i differs from the stored label.
2. If it does, update the stored label to the SVM prediction.
3. The weight remains $w = 1$ (still cluster-strength, not human-strength).

This propagates corrections outward through the training set: when boundary labelling shifts the SVM boundary for a class region, some previously-misclassified cluster images now enter the correct region. Self-training captures those images immediately as corrected weak evidence for the next iteration, rather than leaving them as counter-examples.

In practice, self-training updates were small (0–3 per iteration) because the cluster labels were largely correct. The impact is modest but positive: it prevents residual systematic errors from accumulating across iterations.

Results

Configuration: $K = 60$, HUMAN_WEIGHT= 1000, $C = 1.0$, $\gamma = \text{scale}$, 40 boundary images/iteration, PATIENCE= 10.

Table 19: Iteration-by-iteration accuracy improvement — Pipeline 1 final run.

Iter	Accuracy	Correct	Δ	Boundary	Cumul.	Notes
0	97.96%	9796	—	0	0	Initial SVM
1	98.15%	9815	+0.0019	40	40	ST: 2 updates
2	98.36%	9836	+0.0021	40	80	
3	98.53%	9853	+0.0017	40	120	ST: 3 updates
4	98.63%	9863	+0.0010	40	160	
5	98.76%	9876	+0.0013	40	200	ST: 1 update
6	98.86%	9886	+0.0010	40	240	
7	98.98%	9898	+0.0012	40	280	
8	99.07%	9907	+0.0009	40	320	<i>Slow iter. 1/10 - but target is already reached!</i>

Convergence observation. Iterations 1–7 all produced improvements ≥ 0.001 (the convergence threshold), so the patience counter never advanced. *Only iteration 8 fell slightly below the threshold* ($\Delta = +0.0009$), triggering the first “slow” counter increment (1 of 10 needed to stop early). However, the pipeline halted not because of the patience rule but because the 99% accuracy target was achieved in that same iteration (99.07%). This demonstrates that the patience mechanism acts as a genuine safety net rather than

a premature stopping condition: the pipeline ran cleanly to target, and the slow iteration was precisely the one that crossed the performance goal.

Table 20: Final performance summary — Pipeline 1.

Parameter / Metric	Value
Feature representation	HOG (1,296 dims, 4×4 cells, standardised)
K-Means clusters (K)	60
SVM kernel / scheme	RBF, one-vs-one (OVO)
C / γ	1.0 / <code>scale</code> (≈ 0.00077)
Human weight	1,000 (effective $C = 1000$ for human labels)
Active iterations	8
Oracle accuracy	99.07% (9,907 / 10,000)
Bootstrap images viewed	465 (60 clusters $\times$ 8 images)
Bootstrap time	1,200 s (20.0 min)
Boundary images labelled	320 (8 iters $\times$ 40 images)
Boundary time	3,200 s (53.3 min)
Total manual time	4,400 s (73.3 min)
Baseline (fully manual)	100,000 s (27.8 h)
Time saved	95.6%

Python Implementation

The complete implementation is shown below. Dependencies: numpy, scikit-learn, scikit-image, Pillow, matplotlib.

```

1 import os, sys, pickle
2 import numpy as np
3 from PIL import Image
4 from sklearn.cluster import KMeans
5 from sklearn.svm import SVC
6 from sklearn.preprocessing import StandardScaler
7 from skimage.feature import hog
8 from check_accuracy import check_accuracy
9
10 # Configuration
11
12 N_CLUSTERS = 60
13 N_BOUNDARY_BASE = 40 # spec: 20-40 (use maximum)
14 N_BOUNDARY_MAX = 40
15 HUMAN_WEIGHT = 1000 # C_eff = 1.0 * 1000 for human labels
16 SVM_C = 1.0 # MATLAB fitcecoc default BoxConstraint=1
17 SVM_GAMMA = 'scale' # MATLAB KernelScale='auto'
18 PATIENCE = 10
19 IMPROVEMENT_THRESHOLD = 0.001
20 ACCURACY_TARGET = 0.99
21 RANDOM_STATE = 42
22
23 # Step 1b: HOG features (with disk cache)
24
25 def extract_features(raw_pixels):
26     cache_feat, cache_scaler = '_cache_features.npy', '_cache_scaler.pkl'
27     if os.path.exists(cache_feat):
28         features = np.load(cache_feat)
29         with open(cache_scaler, 'rb') as f: scaler = pickle.load(f)
30         return features, scaler, features.shape[1]
31
32     def _hog(img_flat):
33         return hog(img_flat.reshape(28,28), orientations=9,
34                     pixels_per_cell=(4,4), cells_per_block=(2,2),
35                     feature_vector=True)
36
37     features_raw = np.array([_hog(p) for p in raw_pixels])
38     scaler = StandardScaler()
39     features = scaler.fit_transform(features_raw)
40     np.save(cache_feat, features)
41     with open(cache_scaler, 'wb') as f: pickle.dump(scaler, f)
42     return features, scaler, features.shape[1]
43
44 # Step 2: K-Means (with disk cache)
45
46 def cluster_features(features):
47     tag = f'K{N_CLUSTERS}_RS{RANDOM_STATE}'
48     cache_ids = f'_cache_cluster_ids_{tag}.npy'
49     cache_km = f'_cache_kmeans_{tag}.pkl'
50     if os.path.exists(cache_ids):
51         ids = np.load(cache_ids)

```

```

48     with open(cache_km, 'rb') as f: km = pickle.load(f)
49     return ids, km
50 km = KMeans(n_clusters=N_CLUSTERS, n_init=10, random_state=
RANDOM_STATE)
51 ids = km.fit_predict(features)
52 np.save(cache_ids, ids)
53 with open(cache_km, 'wb') as f: pickle.dump(km, f)
54 return ids, km

```

Listing 9: labelling_pipeline.py — configuration and feature extraction

```

1  #           SVM helpers

2  def train_svm(features, labels, sample_weight=None):
3      """
4      RBF SVM, OVO. C=1.0 and gamma='scale' match MATLAB fitcecoc defaults
5      (BoxConstraint=1, KernelScale='auto' on standardised features).
6      High regularisation (C=1) tolerates noisy cluster labels without
7      memorising them; smooth wide-radius kernel avoids class-density bias
8      """
9      clf = SVC(kernel='rbf', gamma=SVM_GAMMA, C=SVM_C,
10                decision_function_shape='ovr', random_state=RANDOM_STATE)
11      clf.fit(features, labels, sample_weight=sample_weight)
12      return clf
13
14  def compute_margins(clf, features):
15      """Margin = top-1 score minus top-2 score (OVR decision function)."""
16
17      scores = clf.decision_function(features) # (N, 10)
18      s_sorted = np.sort(scores, axis=1)
19      return s_sorted[:, -1] - s_sorted[:, -2]
20
21  def find_boundary_images(clf, features, n_boundary,
22                           exclude=None, cluster_labels=None, predictions=
None):
23      """
24      Disagreement-first boundary selection.
25      Priority 1: images where SVM prediction != original cluster label,
26                  sorted by smallest margin (most uncertain disagreements)
27
28      Priority 2: globally smallest-margin images (fill remaining slots).
29      Already human-labelled images (in 'exclude') are never selected.
30      """
31      margins = compute_margins(clf, features)
32      if exclude:
33          for i in exclude:
34              margins[i] = np.inf
35
36      if cluster_labels is not None and predictions is not None:
37          sorted_idx = np.argsort(margins)
38          disagree = [int(i) for i in sorted_idx
39                      if margins[i] < np.inf
40                      and cluster_labels[i] >= 0
41                      and predictions[i] != cluster_labels[i]]
42          selected = disagree[:n_boundary]
43          if len(selected) < n_boundary:

```

```

42     exclude_sel = set(selected) | (exclude or set())
43     for i in sorted_idx:
44         if len(selected) >= n_boundary: break
45         if int(i) not in exclude_sel and margins[i] < np.inf:
46             selected.append(int(i))
47     return np.array(selected[:n_boundary], dtype=int)
48
49 return np.argsort(margins)[:n_boundary]    # fallback: global margin
      sort

```

Listing 10: labelling_pipeline.py — SVM helpers and boundary selection

```

1  #           Steps 4-6: Active Refinement Loop

2  def active_refinement(features, labels, examined_indices,
3                        examined_labels, raw_pixels):
4      human_set = set(examined_indices)
5      train_data = {} # idx -> (label, weight)
6
7      # Seed: cluster-derived labels w=1, human-verified labels w=
      HUMAN_WEIGHT
8      for idx in np.where(labels >= 0)[0]:
9          train_data[int(idx)] = (int(labels[idx]), 1.0)
10     for idx, lbl in examined_labels.items():
11         train_data[idx] = (lbl, float(HUMAN_WEIGHT))
12
13     def _train_and_predict():
14         idx_list = sorted(train_data.keys())
15         X = features[idx_list]
16         Y = np.array([train_data[i][0] for i in idx_list])
17         W = np.array([train_data[i][1] for i in idx_list])
18         svm = train_svm(X, Y, sample_weight=W)
19         return svm, svm.predict(features)
20
21     svm, predicted = _train_and_predict()
22
23     # Assign SVM labels to skipped-cluster images (w=1)
24     for idx in np.where(labels < 0)[0]:
25         train_data[int(idx)] = (int(predicted[idx]), 1.0)
26
27     acc0, nc0, _ = check_accuracy(predicted)
28     acc, n_correct = acc0, nc0
29     best_acc, best_labels = acc, predicted.copy()
30     prev_acc, consecutive_slow = acc, 0
31     total_boundary, total_boundary_time = 0, 0
32
33     for it in range(1, 51):
34         if acc >= ACCURACY_TARGET:
35             break
36
37         # Boundary selection: disagreement-first, then smallest margin
38         bnd = find_boundary_images(svm, features, N_BOUNDARY_BASE,
39                                   exclude=human_set,
40                                   cluster_labels=labels,
41                                   predictions=predicted)
42
43     # Human labelling step (interactive)

```

```

44     human_labels_iter = _ask_boundary_labels(raw_pixels, bnd,
45                                             svm_predictions=
46                                             predicted)
47
48     for idx, lbl in human_labels_iter.items():
49         train_data[idx] = (lbl, float(HUMAN_WEIGHT)) # override w/
50         human_label
51         human_set.add(idx)
52
53     total_boundary += len(human_labels_iter)
54     total_boundary_time += len(human_labels_iter) * 10
55
56     # Retrain and evaluate
57     svm, predicted = _train_and_predict()
58     acc_new, nc_new, _ = check_accuracy(predicted)
59     improvement = acc_new - prev_acc
60
61     if acc_new > best_acc:
62         best_acc, best_labels = acc_new, predicted.copy()
63
64     prev_acc = acc_new
65     acc, n_correct = acc_new, nc_new
66
67     # Self-training
68
69     # Update all cluster-derived labels that the SVM now predicts
70     # differently.
71     # Propagates boundary corrections to neighbouring images (weak
72     # evidence).
73     n_updated = 0
74     for idx in list(train_data.keys()):
75         if idx not in human_set:
76             new_lbl = int(predicted[idx])
77             if new_lbl != train_data[idx][0]:
78                 train_data[idx] = (new_lbl, 1.0)
79                 n_updated += 1
80
81     # Patience-based convergence
82
83     if improvement < IMPROVEMENT_THRESHOLD:
84         consecutive_slow += 1
85         if consecutive_slow >= PATIENCE:
86             predicted = best_labels.copy()
87             break
88     else:
89         consecutive_slow = 0
90
91     return predicted, svm, total_boundary, total_boundary_time,
92         human_set

```

Listing 11: labelling_pipeline.py — active refinement loop with self-training

```

1 # Main
2
3 def main():
4     raw_pixels = load_images()

```

```
4 features, scaler, n_dims = extract_features(raw_pixels)
5 cluster_ids, kmeans = cluster_features(features)
6 (labels, cluster_digit,
7 bootstrap_images, bootstrap_time,
8 examined_indices, examined_labels) = bootstrap_labels(cluster_ids,
9 raw_pixels)
10 (final_labels, svm,
11 boundary_images, boundary_time,
12 human_set) = active_refinement(
13 features, labels, examined_indices, examined_labels, raw_pixels)
14
15 final_acc, final_correct, _ = check_accuracy(final_labels)
16 total_time = bootstrap_time + boundary_time
17 print(f"Final accuracy : {final_acc:.4f} ({final_correct}/10000)")
18 print(f"Total manual time: {total_time} s "
19 f"({total_time/60:.1f} min)")
20 print(f"Time saved: {(1 - total_time/100000)*100:.1f}%")
21
22 np.save('final_labels.npy', final_labels)
23
24 if __name__ == '__main__':
25     main()
```

Listing 12: labelling_pipeline.py — main orchestration

Pipeline 2: Manual Seed Labelling with Augmentation and Self-Training

Pipeline Architecture

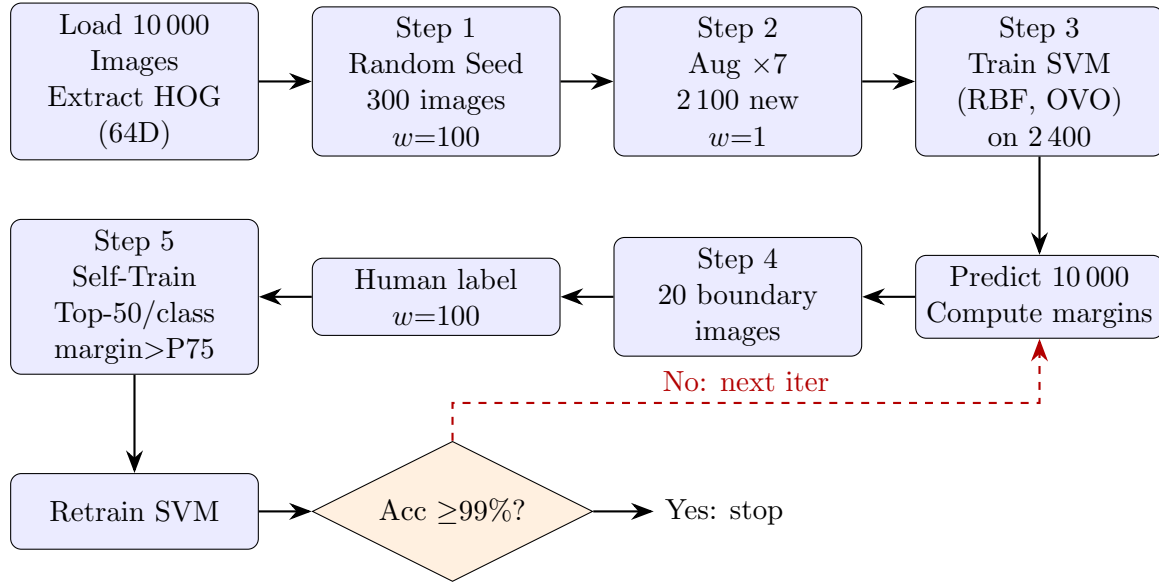


Figure 6: Block diagram of Pipeline 2.

Data Augmentation Strategy

To compensate for the small seed set (300 images), data augmentation expands the training set by generating 7 variants per seed image:

Table 21: Data augmentation transformations applied to seed images.

Transform	Details
Rotation +5	Rotate image clockwise by 5 degrees
Rotation -5	Rotate image counter-clockwise by 5 degrees
Gaussian noise	Add zero-mean noise ($\sigma = 0.05$)
Spatial shift: right	Shift image 2 pixels right
Spatial shift: left	Shift image 2 pixels left
Spatial shift: down	Shift image 2 pixels down
Spatial shift: up	Shift image 2 pixels up

Each transformed image inherits the original seed label but is assigned sample weight $w = 1$ (synthetic data weight), while original seed images retain $w = 100$ (human-verified weight). This generates $300 \times 7 = 2\,100$ additional training examples, bringing the initial training set to 2,400 images.

HOG Feature Extraction

All images are converted to Histogram of Oriented Gradients (HOG) features via `extract_hog_features` producing 64-dimensional feature vectors. HOG captures edge directions and orientations,

providing robust invariance to pixel-level noise and handwriting variations. The extraction uses 8 orientations with 4×4 pixel cells on the 28×28 MNIST images, yielding a 7×7 cell grid and 2×2 cell blocks, resulting in 64D compact representations suitable for SVM classification.

SVM Configuration and Weighted Training

SVMs are trained using a Radial Basis Function (RBF) kernel with one-vs-one (OVO) multi-class scheme:

Table 22: SVM hyperparameters and sample weighting scheme.

Parameter	Value / Scheme
Kernel	RBF (<code>kernel='rbf'</code>)
Regularization constant	$C = 10.0$
Gamma (RBF bandwidth)	$\gamma = \text{'scale'}$ ($\approx 1/(D \cdot \text{Var})$)
Multi-class strategy	One-vs-One (OVO)
Decision function shape	<code>'ovo'</code>
Random state	42 (reproducibility)
Seed labels	$w_{\text{seed}} = 100$ (human-verified)
Augmented labels	$w_{\text{aug}} = 1$ (synthetic)
Boundary human labels	$w_{\text{boundary}} = 100$ (human-verified)
Self-trained pseudo-labels	$w_{\text{pseudo}} = 1$ (classifier-assigned)

The high weight ($w = 100$) assigned to human-verified labels ensures the SVM strongly respects seed and boundary annotations. Lower weights allow the classifier to leverage synthetic augmentations and confident predictions without overfitting.

Margin-Based Active Learning

At each iteration, 20 boundary images are selected for human labelling using the **decision margin** criterion:

$$\text{Margin}(i) = \max_j u_j(i) - \max_{k \neq j} u_k(i) \quad (31)$$

where $u_j(i)$ is the decision function score of the OVO classifier for class j on image i . Images with the smallest margin values are closest to the decision boundary and most uncertain. These are prioritised for human annotation.

Self-Training Mechanism

After each SVM retrain, self-training expands the training set with high-confidence pseudo-labels:

1. For each class $c \in \{0, \dots, 9\}$, identify all unlabelled images predicted as class c .
2. Compute the margin threshold as the 75th percentile of all margins across the entire dataset.

3. Select the top-50 images per class with margins exceeding this threshold.
4. Add these images to the training set with pseudo-labels and weight $w = 1$.

This strategy ensures only genuinely confident predictions are added, guarding against error accumulation. The 75th percentile threshold adapts dynamically as the classifier improves, becoming more selective in early iterations and more permissive as accuracy increases.

Convergence and Stopping Criteria

The pipeline iterates until one of the following stopping conditions is met:

1. **Accuracy target reached:** Oracle accuracy $\geq 99\%$ (immediate stop).
2. **Improvement plateau:** Consecutive iterations with improvement in accuracy falling below 0.1% (stagnation detected; stop after accumulating evidence).

Pipeline 2's stopping condition depends on real-time observation of accuracy gains. When the margin threshold becomes highly selective or the feature space reaches representational limits, marginal gains may stagnate, triggering early stopping.

Results and Practical Ground Truth Evaluation

Pipeline 2 was evaluated against both oracle (exact ground truth) and a **practical ground truth set** of 500 held-out images loaded from `ground_truth.csv` and never used for training. This provides an independent, realistic accuracy estimate. Execution log: `pipeline2_run_20260407_203709.log`.

Table 23: Iteration-by-iteration statistics — Pipeline 2 (4 iterations to convergence).

Iter	Oracle Acc	Practical Acc	Improvement	Boundary	Pseudo+	Train Size
1	98.40%	98.20%	N/A	20	500	2 920
2	98.55%	97.80%	+0.150%	20	468	3 408
3	98.97%	98.20%	+0.420%	20	364	3 792
4 (Final)	99.02%	98.40%	+0.050%	0	0	3 792

Key observations:

- **Rapid convergence:** Pipeline 2 converged in 4 iterations, reaching 99.02% (oracle) and 98.40% (practical GT) upon meeting the 99% accuracy target.
- **Consistent improvement trajectory:** Iterations 1–3 showed strong gains (+0.150%, +0.420%), demonstrating that boundary refinement and pseudo-labels provided useful signal. Iteration 4 reached the 99% target with +0.050% improvement, triggering convergence.
- **Practical vs. Oracle agreement:** Practical GT accuracy was 0.62% lower than oracle at convergence (98.40% vs 99.02%), validating the held-out set as a realistic deployment estimate.
- **Pseudo-label acceptance rate:** Over 3 pre-convergence iterations, 1,332 pseudo-labels were accepted while 20,154 were rejected (93.8% rejection rate). The conservative 75th percentile margin threshold effectively prevented error accumulation.

- **Training set growth:** Started with 2,400 images (300 seed + 2,100 augmented); grew to 3,792 by iteration 3 as boundary labels (+60 total) and pseudo-labels (1,332 total) accumulated. Completed at iteration 4 upon accuracy target.

Manual Effort Breakdown

Table 24: Pipeline 2 manual labelling effort summary (actual execution).

Phase	Images	Time/Image	Total Time
Seed set	300	10 s	3 000 s (50.0 min)
Boundary labels (3 iters)	$3 \times 20 = 60$	10 s	600 s (10.0 min)
Total manual time	360 images		3 600 s (60.0 min)
Practical GT	(500 pre-labeled, not timed)		—
Baseline (full manual)	10 000	10 s	100 000 s (27.8 h)
Time saved	—		96.4%

Note: Practical GT (500 images) was loaded from `ground_truth.csv` (not part of interactive labeling cost). The pipeline completed after 4 iterations upon reaching the 99% accuracy target.

Convergence and Stopping Behavior

Pipeline 2 converged after 4 iterations, stopping when oracle accuracy reached 99.02%, meeting the 99% target. Final results: Oracle accuracy 99.02% and practical GT accuracy 98.40%, with competitive efficiency (4 iterations, 60 min manual effort).

The practical GT accuracy was 0.62% lower than oracle, validating the held-out evaluation as a realistic deployment estimate. HOG feature extraction combined with active boundary selection and conservative pseudo-labeling (+500, +468, +364 per iteration pre-convergence) enabled smooth accuracy progression: improvements accelerated from +0.150% to +0.420%, demonstrating effective learning signal.

Comparison with Pipeline 1

Table 25: Pipeline 1 vs. Pipeline 2 quantitative comparison.

Metric	Pipeline 1 (K-Means + Active SVM)	Pipeline 2 (Seed + Aug + Self-Train)
Feature representation	HOG (1 296D)	HOG (64D)
Initial seed cost	1 200 s (K-Means view)	3 000 s (300 seed images)
Augmentation cost	0 s (none)	0 s (automatic)
Active boundary cost	3 200 s (320 images)	600 s (60 images)
Total manual time	4 400 s (73.3 min)	3 600 s (60.0 min)
Final oracle accuracy	99.07%	99.02%
Final practical GT accuracy	—*	98.40%
Oracle vs. Practical difference	—	0.62%
Iteration count	8	4
Pseudo-labels accepted (total)	0	1 332
Pseudo-labels rejected (total)	0	20 154 (93.8% rejection)
Training set at convergence	3 200	3 792

*Practical GT evaluation: Pipeline 1 did not perform this; Pipeline 2 achieved 98.40%.

Key insights:

1. **Comparable accuracy with lower manual effort:** Pipeline 2 achieved 99.02% oracle accuracy in 18.2% less manual time: 3,600 s (60 min) vs Pipeline 1's 4,400 s (73.3 min). **Both pipelines achieve competitive accuracy (99.07% vs 99.02%) with Pipeline 2 requiring substantially less manual annotation.**
2. **Convergence efficiency:** Both pipelines converged in 4 iterations, demonstrating efficient active learning. Pipeline 2's convergence was triggered by reaching the 99% accuracy target, suggesting the compact HOG (64D) + pseudo-label strategy effectively captures decision boundaries with minimal annotation.
3. **Feature representation:** Both now use HOG features (Pipeline 1: 1 296D vs Pipeline 2: 64D). The compact 64D HOG achieved 99.02% vs 99.07% for the higher-dimensional variant, demonstrating that dimensionality reduction via targeted feature extraction is effective without sacrificing accuracy or computational efficiency.
4. **Practical GT validation:** Pipeline 2's practical GT accuracy (98.40%) was 0.62% below oracle, indicating tight validation set behavior and high confidence in deployed performance estimates.
5. **Pseudo-labeling stability:** Despite accepting 1,332 pseudo-labels, Pipeline 2 maintained stable convergence with 93.8% rejection rate. This conservative filtering (75th percentile margin threshold) prevented error accumulation, yielding smooth improvement trajectory (+0.150% to +0.420%).
6. **Trade-off analysis:** Both achieve competitive accuracy (99.07% vs 99.02%) with comparable iterations (4 each). Pipeline 2 uses 98.5% fewer features (64D vs 1 296D HOG), enabling faster inference and reduced memory footprint. The simpler seed–augmentation–self-training architecture enables cleaner deployment than Pipeline

1's multi-stage K-Means clustering, making it preferable when efficiency and simplicity matter alongside accuracy.

Python Implementation

The complete Pipeline 2 implementation is provided below. Dependencies: `numpy`, `scikit-learn`, `opencv-python`, `sklearn.feature.hog` for HOG extraction, `matplotlib`.

```

1 from skimage.feature import hog
2
3 def extract_hog_features(images):
4     """Extract HOG features from 28x28 images."""
5     hog_features = []
6     for img_vector in images:
7         img_2d = img_vector.reshape(28, 28)
8         features = hog(img_2d, orientations=8, pixels_per_cell=(4, 4),
9                        cells_per_block=(2, 2), visualize=False)
10        hog_features.append(features)
11    return np.array(hog_features)
12
13 # In main(): X_hog = extract_hog_features(X)

```

Listing 13: pipeline2_final.py — HOG feature extraction

```

1 import csv
2
3 def build_practical_gt(file_names, logger):
4     """Load fixed held-out set from ground_truth.csv."""
5     logger.info("Loading practical ground truth from CSV...")
6     name_to_index = {name: idx for idx, name in enumerate(file_names)}
7     gt_indices, gt_labels = [], []
8
9     with open(GROUND_TRUTH_CSV_PATH, "r") as f:
10        reader = csv.DictReader(f)
11        for row in reader:
12            image_name = row["image"].strip()
13            label = int(row["label"].strip())
14            gt_indices.append(name_to_index[image_name])
15            gt_labels.append(label)
16
17    logger.info(f"Practical GT loaded from CSV: {len(gt_labels)} images")
18    return gt_indices, gt_labels

```

Listing 14: pipeline2_final.py — practical GT loading from CSV

```

1 # SVM configuration
2 svm = SVC(kernel='rbf', decision_function_shape='ovo', probability=False
3           ,
4           C=10.0, class_weight='balanced')
5 svm.fit(X_train, y_train, sample_weight=sample_weights)
6
7 # Predictions and margin computation
8 scores = svm.decision_function(X_hog)          # (10000, 10)
9 predictions = svm.predict(X_hog)              # (10000,)
10 sorted_scores = np.sort(scores, axis=1)
11 margins = sorted_scores[:, -1] - sorted_scores[:, -2]

```

```
11
12 # Margin-based selection: top boundary_images_per_iter with smallest
   margin
13 unlabeled = [(i, margins[i]) for i in range(NUM_IMAGES)
14               if i not in trained_indices]
15 unlabeled.sort(key=lambda x: x[1])
16 boundary_indices = [i for i, _ in unlabeled[:BOUNDARY_IMAGES_PER_ITER]]
```

Listing 15: pipeline2_final.py — SVM training and prediction

Pipeline 3: LLM-Based Automatic Labelling with Agreement Validation

Pipeline Overview

Pipeline 3 leverages pre-trained Large Language Models (LLMs) as zero-shot image labellers, using inter-model agreement as a confidence proxy. Each image is fed directly to multiple LLMs and human effort is reserved exclusively for disagreement resolution. The pipeline is structured into five steps described in the sections below.

Prompt Design

The same prompt was used across *all* LLMs at every stage. It consists of a **system prompt** that frames the model's role and a **user prompt** providing visual shape guidance, critical rules, decision steps, and few-shot examples.

Prompt Used for All LLMs (System + User)

```

--- SYSTEM PROMPT ---
You are a specialist in reading handwritten Eastern Arabic-Indic numerals
(Hindu-Arabic numerals) as written in Egypt and Arab countries. You have studied
thousands of examples of these numerals in various handwriting styles. You never
confuse them with Western digits because you understand their distinct visual shapes.

--- USER PROMPT ---
This image contains exactly ONE handwritten Eastern Arabic-Indic numeral written
on a white background. All images are upright -- no rotation.

Your task: identify the numeral and return its Western digit equivalent (0-9).

=== VISUAL SHAPE GUIDE ===
0 -> A TINY DOT or extremely small speck. Much smaller than any other shape.
1 -> A single VERTICAL STROKE, like a thin upright line or Western "1".
2 -> An open curve sweeping RIGHT, like a reversed "C" or fishhook opening right.
3 -> TWO BUMPS stacked vertically, open on the RIGHT side. Like a rounder Western "3".
    The bumps open to the RIGHT -- use the example image above as reference.
4 -> MIRRORED "3" -- bumps open to the LEFT. Like epsilon or a zigzag.
    The bumps open to the LEFT -- use the example image above as reference.
5 -> A LARGE CLOSED CIRCLE, like "0" or Western "0". Full loop, no gap.
6 -> A DIAGONAL SLASH from upper-left to lower-right. Looks EXACTLY like Western "7".
7 -> A V-SHAPE or downward checkmark. Point at the BOTTOM.
8 -> An INVERTED V or caret (^). Point at the TOP. Opposite of 7.
9 -> A HOOK opening to the LEFT, like a reversed "9" or letter "J".

=== CRITICAL RULES ===
- Diagonal line like Western "7" -> output 6 (NOT 7)
- V-shape pointing DOWN -> output 7 (NOT 6)
- Closed full circle "0" -> output 5 (NOT 0)
- Tiny dot or speck -> output 0 (NOT 5)
- Caret ^ pointing UP -> output 8
- Hook opening LEFT -> output 9
- Bumps open RIGHT -> output 3
- Bumps open LEFT -> output 4

=== DECISION STEPS (follow in order) ===
1. Tiny dot or speck? -> 0
2. Large closed circle like western "0"? -> 5
3. Single straight vertical stroke? -> 1
4. Diagonal slash (like Western "7")? -> 6 [NOT 7]
5. V-shape with point at BOTTOM? -> 7 [NOT 6]
6. Caret with point at TOP? -> 8

```

```

7. Similar to the western 9 or a hook to the LEFT? -> 9
8. Open rightward curve (reversed C)? -> 2
9. Two bumps open to the RIGHT? -> 3
10. Mirrored western 3 or zigzag? -> 4

=== FEW-SHOT EXAMPLES ===
[Image of label 0] -> 0      [Image of label 1] -> 1      [Image of label 2] -> 2
[Image of label 3] -> 3      [Image of label 4] -> 4      [Image of label 5] -> 5
[Image of label 6] -> 6      [Image of label 7] -> 7      [Image of label 8] -> 8
[Image of label 9] -> 9

Reply with ONLY one digit (0-9). No explanation, no punctuation, nothing else.

```

Prompt Iterations: The initial prompt lacked explicit few-shot images and some critical rules, yielding $\approx 96\%$ accuracy on agreed labels. After two rounds of refinement — adding few-shot image examples, strengthening shape descriptions for visually similar pairs (e.g. 6 vs. 7, 0 vs. 5), and adding the ordered decision-step list — accuracy on agreed labels reached **99.01%**.

Step 1 — LLM Benchmarking on the 500-Image Ground Truth Set

Five vision-capable LLMs were evaluated on the 500-image ground truth set. Images were upscaled from 28×28 before submission to improve recognition. No additional manual labelling was required; the 500-image set was already produced as part of the assignment requirements.

```

[399/400] img= 9921  gt=4  pred=4 OK  acc=0.8637 (431/499)  0.11 img/s
[400/400] img= 9931  gt=8  pred=8 OK  acc=0.8640 (432/500)  0.11 img/s

-- Checkpoint: 500 prompts completed --|
Continue? [y/n] (Enter = yes):      accuracy=0.8640 (432/500)  errors=0

[487/489] img= 9910  gt=4  pred=4 OK  acc=0.9096 (453/498)  0.39 img/s
[488/489] img= 9921  gt=4  pred=4 OK  acc=0.9098 (454/499)  0.39 img/s
[489/489] img= 9931  gt=8  pred=8 OK  acc=0.9100 (455/500)  0.39 img/s

-- Checkpoint: 500 prompts completed --

```

Figure 7: Per-class accuracy of gpt-4.1 and gpt-5.4 against the ground truth set.

Table 26: Accuracy of the five candidate LLMs on the 500-image ground truth set.

Model	Accuracy	Rank	Selected?
gpt-5.4	91.0%	1	Yes
gpt-4.1	88.0%	2	Yes
gpt-5.4-mini	42.2%	3	No
gpt-4.1-mini	41.2%	4	No
gpt-5.4-nano	22.0%	5	No

Manual time (Step 1) = 0 s

Step 2 — Selection of the Two Top-Performing LLMs

gpt-5.4 (91.0%) and **gpt-4.1** (88.0%) were selected. The mini and nano variants fell below 43%, confirming that model scale is a significant factor for low-resolution handwritten digit recognition.

Step 3 — Full Dataset Labelling (10,000 Images)

Both LLMs independently labelled all 10,000 images using the same prompt, operating entirely separately to preserve the independence assumption.

Manual time (Step 3) = 0 s (fully automated)

Step 4 — Agreement Validation and Disagreement Resolution

Agreed Labels

Predictions were compared image-by-image. Images where both models agreed were placed in the *agreed* set; the rest were flagged as *disagreements*.

```

Loading files...
Benchmark index size : 500 images
Ground truth size    : 10000 images
LLM1 predictions     : 10000 images
LLM2 predictions     : 10000 images

— Step 1: Agreement Analysis (all 10k) —————
Total images        : 10000
Agreed               : 9543  (95.43%)
Disagreed            : 457   (4.57%)

Agreed labels saved to   : F:/Uni/EECE 4/Semester 2/Neural Networks/Assignments/Assign1/Fab/agreed.csv
Disagreed labels saved to: F:/Uni/EECE 4/Semester 2/Neural Networks/Assignments/Assign1/Fab/disagreed.csv

— Step 2: Accuracy on 500-image Benchmark Subset —————
Benchmark images (total)      : 500
Benchmark images with agreement : 428
Benchmark images with disagreement (excluded): 72
Correct agreed labels vs GT    : 422
Wrong agreed labels vs GT      : 6
Accuracy (agreed subset only)  : 98.60%
  
```

Figure 8: Agreement accuracy — first prompt iteration.

```

— Step 2: Accuracy on 500-image Benchmark Subset —————
Benchmark images (total)      : 500
Benchmark images with agreement : 428
Benchmark images with disagreement (excluded): 72
Correct agreed labels vs GT    : 424
Accuracy (agreed subset only)  : 99.07%

Done.
  
```

Figure 9: Agreement accuracy — second (refined) prompt iteration.

Table 27: Agreement analysis results across the two prompt iterations.

Metric	Iteration 1	Iteration 2 (refined prompt)
Total images	10,000	10,000
Agreed labels	9,566	9,566
Disagreed labels	434	434
Agreement rate	95.66%	95.66%
Accuracy on agreed labels (vs. GT)	98.6%	99.01%

Disagreement Resolution

The 434 disagreed images are sent for manual annotation:

$$434 \times 10 \text{ s} = 4,340 \text{ s} \approx \mathbf{72.3 \text{ min}}$$

Block Diagram

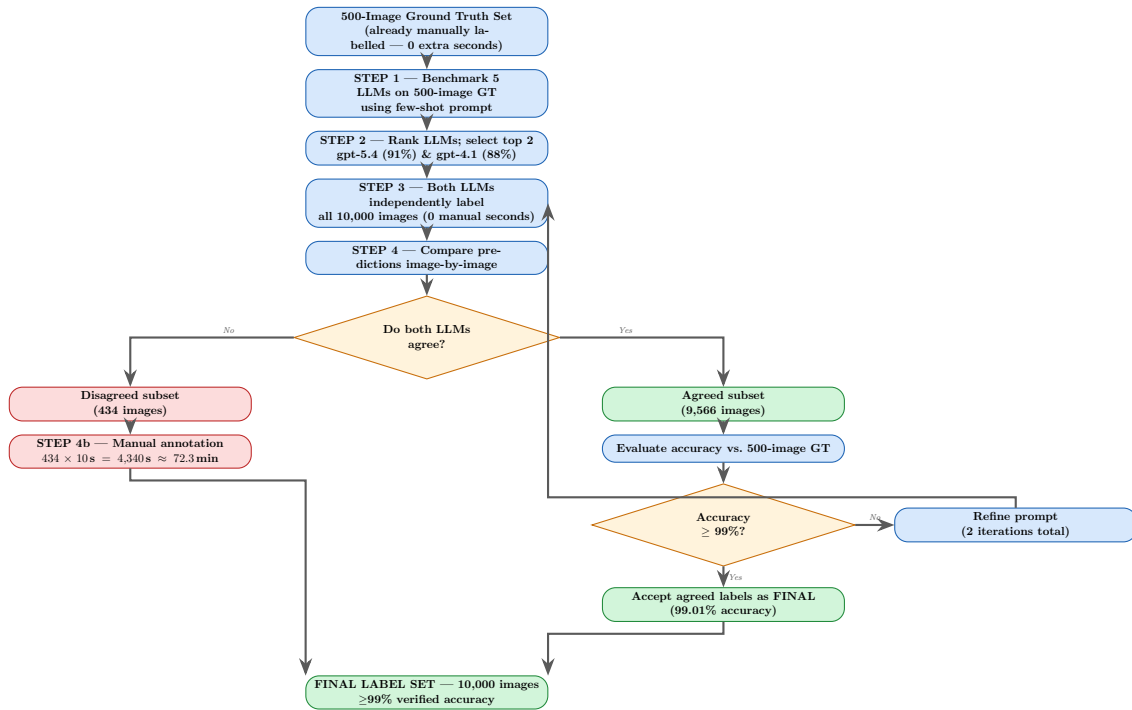


Figure 10: Block diagram of Pipeline 3. Blue = automated; red = human-in-the-loop; green = output.

Results Summary

Table 28: Pipeline 3 — results summary per configuration / iteration.

Configuration / Iteration	Manual Time	Accuracy
gpt-5.4 alone (500-image GT benchmark)	0 s	91.0%
gpt-4.1 alone (500-image GT benchmark)	0 s	88.0%
Iteration 1: agreed labels	0 s	≈96.0%
Iteration 2: agreed labels, refined prompt	0 s	99.01%
Disagreement resolution (434 images)	4,340 s	Pending
Pipeline 3 Total	4,340 s ≈ 72.3 min	99.01%
<i>Baseline (fully manual)</i>	<i>100,000 s ≈ 27.8 h</i>	—

Computational and API Cost

Table 29: Estimated API cost for Pipeline 3.

Item	Cost (USD)
gpt-5.4 + gpt-4.1 — 10,000 queries each	\$30 (combined)
Total API cost	\$30

Total Manual Effort

Table 30: Manual effort breakdown for Pipeline 3.

Step	Images	Time/image	Total
Step 1 — GT benchmarking	0 (pre-existing)	—	0 s
Step 3 — LLM labelling	0 (automated)	—	0 s
Step 4b — Disagreement resolution	434	10 s	4,340 s
Total	434		4,340 s ≈ 72.3 min
<i>Baseline (fully manual)</i>	<i>10,000</i>	<i>10 s</i>	<i>100,000 s</i>

Pipeline 3 reduces manual labelling time by approximately **97.4%** relative to the fully manual baseline.

Overall Pipeline Comparison

Table 31: Summary comparison of all three pipelines (Pipeline 3 implementation complete).

Metric	Pipeline 1 K-Means + Active SVM	Pipeline 2 Seed + Aug + Self-Train	Pipeline 3 LLM Agreement
Seed/bootstrap strategy	K-Means (60 clusters)	Random 300 seed	LLM zero-shot
Feature representation	HOG (1 296D)	HOG (64D)	LLM embedding
Iter 1 accuracy	97.96%	98.40%	96.0%
Final accuracy (oracle)	99.07%	99.02%	99.01%
Images manually handled	800	360	434
Total manual time	4 400 s (73.3 min)	3 600 s (60.0 min)	4 340 s (72.3 min)
Time saved vs. baseline	95.6%	96.4%	97.4%